

Project Context: iot-fire-detection-ai

Project root: /mnt/e/projects/iot-fire-detection-ai

This PDF is structured for LLM ingestion: file paths are explicit and text blocks are wrapped in code fences.

1. Project Tree

```
iot-fire-detection-ai/
├── ai/
│   ├── confusion_matrix.png
│   ├── dataset_fire_detection.csv
│   ├── evaluation_results.txt
│   ├── model_fire_detection.joblib
│   ├── predict.py
│   └── train_model.py
├── article_screenshots/
│   ├── 01_simulator_mqtt_publish.png
│   ├── 02_mqtt_subscriber_received_json.png
│   ├── 03_node_red_final_flow.png
│   ├── 04_node_red_debug_normal.png
│   ├── 05_node_red_debug_suspect.png
│   ├── 06_node_red_debug_danger_alert.png
│   ├── 07_ai_prediction_tests.png
│   ├── 08_confusion_matrix.png
│   ├── 09_evaluation_results.txt
│   ├── dashboard_danger.png
│   ├── dashboard_normal.png
│   ├── dashboard_suspect.png
│   └── thingsboard_dashboard.png
├── cloud/
│   ├── preuve_test_bout_en_bout.md
│   └── README_demo.md
├── docs/
│   ├── checklist_soutenance.md
│   ├── mqtt_format_membre1.md
│   ├── README_node_red_base(membre2).md
│   ├── README_task3.md
│   └── validation_membre1.md
├── flow/
│   ├── flow_node_red_base.json
│   └── integration_ia_node_red/
│       ├── screenshots_tache4/
│       │   ├── capture_alert_danger.png
│       │   ├── capture_flow_node_red_ai_alert.png
│       │   ├── capture_mqtt_publish_simulator.png
│       │   ├── capture_prediction_normal.png
│       │   └── capture_prediction_suspect.png
│       ├── flow_node_red_ai_alert.json
│       └── README_integration_ia.md
├── logs/
│   └── sample_output.txt
├── screenshots/
│   ├── capture_node_red_reception.png
│   ├── danger_alert.png
│   ├── dashboard_danger.png
│   ├── dashboard_normal.png
│   ├── dashboard_suspect.png
│   ├── normal_alert.png
│   ├── suspect_alert.png
│   └── thingsboard_dashboard.png
├── simulator/
│   ├── config.json
│   └── simulate_fire_sensors.py
├── tests/
│   └── mqtt_subscriber_test.py
├── .gitignore
├── config.example.json
├── iot-fire-detection-ai.pdf
├── PROJECT_STATUS.md
├── README.md
└── requirements.txt
```

2. Source And Text Files

.gitignore

Type: text | Source: /mnt/e/projects/iot-fire-detection-ai/.gitignore

```
FILE: .gitignore
```text
__pycache__/
*.pyc
.env
venv/
.venv/
node_modules/
.DS_Store
.pytest_cache/
```
```

config.example.json

Type: text | Source: /mnt/e/projects/iot-fire-detection-ai/config.example.json

```
FILE: config.example.json
```json
{
 "mqtt": {
 "broker_host": "localhost",
 "broker_port": 1883,
 "sensor_topic": "iot/fire/sensor/data",
 "alert_topic": "iot/fire/alert"
 },
 "thingsboard": {
 "host": "thingsboard.cloud",
 "mqtt_port": 1883,
 "telemetry_topic": "v1/devices/me/telemetry",
 "access_token": "PUT_YOUR_DEVICE_TOKEN_HERE"
 },
 "ai": {
 "predict_script_path": "ai/predict.py"
 }
}
```
```

PROJECT_STATUS.md

Type: text | Source: /mnt/e/projects/iot-fire-detection-ai/PROJECT_STATUS.md

```
FILE: PROJECT_STATUS.md
```markdown
Statut du projet IoT et IA

Synthèse

| Composant | Statut | Preuve | Notes |
|---|---|---|---|
| Simulateur capteurs | Implémente | `simulator/simulate_fire_sensors.py` | Génère des scénarios normal, suspect et danger |
| Broker MQTT local | À configurer localement | `simulator/config.json` | Démonstration prévue sur `localhost:1883` |
| Flow Node-RED de base | Implémente | `flow/flow_node_red_base.json` | Réception et traitement initial des messages |
| Dataset IA | Disponible | `ai/dataset_fire_detection.csv` | Base utilisée pour l'entraînement |
| Entraînement IA | Réalisé | `ai/evaluation_results.txt` | Évaluation et modèle déjà produits |
| Prédiction IA | Implémente | `ai/predict.py` | Retourne `ai_status`, `risk_score` et `alert` |
| Intégration IA Node-RED | Implémente | `integration_ia_node_red/flow_node_red_ai_alert.json` | Appel de `predict.py` et f |
| Topic alerte MQTT | Implémente | `integration_ia_node_red/flow_node_red_ai_alert.json` | Publication sur `iot/fire/alert` |
| Dashboard ThingsBoard | Documenté | `screenshots/` | Captures de tableaux de bord et états |
| Test bout en bout | Documenté | `cloud/preuve_test_bout_en_bout.md` | Preuve académique de la chaîne complète |

Configuration requise avant exécution

- installer les dépendances avec `pip install -r requirements.txt`
- démarrer Mosquitto avec `sudo service mosquitto start`
- configurer le chemin `predict.py` dans Node-RED si nécessaire
- configurer le token ThingsBoard avant l'envoi de télémétrie
```
```

README.md

Type: text | Source: /mnt/e/projects/iot-fire-detection-ai/README.md

```
FILE: README.md
```markdown
Système intelligent de détection d'incendie basé sur l'IoT et l'IA

Présentation du projet

Ce projet assemble un système complet de démonstration académique pour la détection d'incendie. Un simulateur Python génère

Architecture globale

```text
Simulateur Python -> MQTT Mosquitto -> Node-RED -> IA predict.py -> Alerte MQTT -> ThingsBoard
```

Structure du projet

Dossier	Description
`simulator/`	Simulateur de capteurs incendie et configuration MQTT locale
`flow/`	Flow Node-RED de base
`ai/`	Dataset, entraînement, modèle et script de prédiction IA
`integration_ia_node_red/`	Flow Node-RED avec intégration IA et documentation associée
`cloud/`	Guide de démonstration cloud et preuve de test bout en bout
`screenshots/`	Captures d'écran de validation et de tableau de bord
`docs/`	Documentation complémentaire, formats MQTT et checklist
`tests/`	Scripts de test et de vérification MQTT

Technologies utilisées

Le projet utilise Python, MQTT, Mosquitto, Node-RED, JSON, Scikit-learn, Random Forest et ThingsBoard.

Installation

```bash
pip install -r requirements.txt
sudo apt install mosquitto mosquitto-clients
```

Configuration MQTT locale

La configuration locale par défaut repose sur :

- broker MQTT : `localhost:1883`
- topic capteurs : `iot/fire/sensor/data`
- topic alertes : `iot/fire/alert`

Des brokers publics peuvent être utilisés de manière optionnelle pour des essais distants, mais ils ne sont pas requis pour

Exécution de la démonstration

1. Démarrer Mosquitto :
```bash
sudo service mosquitto start
```

2. Démarrer Node-RED :
```bash
node-red
```

3. Importer puis déployer le flow Node-RED.
4. Démarrer le simulateur :
```bash
python simulator/simulate_fire_sensors.py
```

5. Observer le topic MQTT capteur :
```bash
mosquitto_sub -h localhost -t iot/fire/sensor/data
```

6. Observer la sortie debug Node-RED et le tableau de bord ThingsBoard.

Tests rapides

```bash
python simulator/simulate_fire_sensors.py --once
python simulator/simulate_fire_sensors.py --offline --once
python ai/predict.py 24 80 120 55
python ai/predict.py 48 340 300 40
python ai/predict.py 78 850 720 22
```

Sorties IA attendues :

- `normal` -> `{"ai_status": "normal", "alert": false}`
```

```
- `suspect` -> `{"ai_status": "suspect", "alert": false}`
- `danger` -> `{"ai_status": "danger", "alert": true}`

Différence entre pre_alert, ai_status et alert

- `pre_alert` = première estimation basée sur des seuils dans Node-RED.
- `ai_status` = classification finale produite par `predict.py`.
- `alert` = `true` uniquement lorsque `ai_status == "danger"`.
```

Exemple de JSON enrichi :

```
```json
{
  "device_id": "fire_sensor_01",
  "timestamp": "2026-05-31T15:40:00",
  "temperature": 78,
  "smoke": 850,
  "gas": 720,
  "humidity": 22,
  "location": "Salle_1",
  "scenario": "danger",
  "pre_alert": "danger",
  "ai_status": "danger",
  "risk_score": 1.0,
  "alert": true
}
```
```

### ## Configuration ThingsBoard

Chaque utilisateur doit configurer son propre token de device ThingsBoard ainsi que son tableau de bord.

### ## État actuel du projet

Éléments déjà implémentés :

- simulateur
- publication MQTT locale
- flow Node-RED de base
- modèle IA
- intégration IA dans Node-RED
- topic MQTT d'alerte
- captures et preuves ThingsBoard

Éléments à configurer localement :

- service Mosquitto
- chemin `predict.py` dans Node-RED si nécessaire
- token ThingsBoard

### ## Problèmes fréquents et corrections

- `paho-mqtt` non installé : exécuter `pip install -r requirements.txt`
- Mosquitto non démarré : exécuter `sudo service mosquitto start`
- mauvais chemin d'exécution Node-RED : ajuster la commande du node `Call predict.py`
- token ThingsBoard absent : configurer le token du device dans le flow concerné
- topic MQTT incohérent : vérifier `iot/fire/sensor/data` et `iot/fire/alert`

...

## requirements.txt

Type: text | Source: /mnt/e/projects/iot-fire-detection-ai/requirements.txt

```
FILE: requirements.txt
```text
paho-mqtt>=1.6.1
pandas
numpy
scikit-learn
matplotlib
seaborn
joblib
```
```

## ai/dataset\_fire\_detection.csv

Type: text | Source: /mnt/e/projects/iot-fire-detection-ai/ai/dataset\_fire\_detection.csv

```
FILE: ai/dataset_fire_detection.csv
```

```
```text
temperature,smoke,gas,humidity,label
22,60,80,58,normal
24,80,120,55,normal
20,50,90,60,normal
23,70,100,57,normal
25,90,110,54,normal
21,55,85,59,normal
26,75,95,56,normal
19,45,75,62,normal
27,85,105,53,normal
... [truncated to first 10 rows]
```
```

## ai/evaluation\_results.txt

Type: text | Source: /mnt/e/projects/iot-fire-detection-ai/ai/evaluation\_results.txt

```
FILE: ai/evaluation_results.txt
```text

=====
EVALUATION DU MODELE IA - Detection d incendie
Tache 3 - Bourti Ayoub
=====

Modele      : Random Forest Classifier
Dataset     : dataset_fire_detection.csv
Echantillons : 120 total | 90 train | 30 test
Features    : temperature, smoke, gas, humidity
Classes     : normal, suspect, danger

-----

ACCURACY SUR LE JEU DE TEST
-----

Accuracy      : 100.00 %

-----

CROSS-VALIDATION (5-Fold Stratifie)
-----

Scores       : ['100.00%', '100.00%', '100.00%', '100.00%', '100.00%']
Moyenne      : 100.00 %
Ecart-type   : 0.00 %

-----

RAPPORT DE CLASSIFICATION
-----

              precision    recall  f1-score   support

    danger         1.00        1.00        1.00         10
    normal         1.00        1.00        1.00         10
    suspect         1.00        1.00        1.00         10

 accuracy                   1.00         30
 macro avg          1.00        1.00        1.00         30
weighted avg          1.00        1.00        1.00         30

-----

MATRICE DE CONFUSION (lignes=reel, colonnes=predit)
-----

Classes      : normal, suspect, danger
[[10  0  0]
 [ 0 10  0]
 [ 0  0 10]]

-----

IMPORTANCE DES FEATURES
-----

smoke          0.310065
temperature    0.282858
gas            0.277583
humidity       0.129495

-----

TESTS SUR LES 3 SCENARIOS DU PROJET
-----

Scenario 1 - Normal : temp=24 smoke=80 gas=120 humid=55
Prediction = normal (attendu: normal)

Scenario 2 - Suspect : temp=48 smoke=340 gas=300 humid=40
Prediction = suspect (attendu: suspect)

Scenario 3 - Danger : temp=78 smoke=850 gas=720 humid=22
```

```
Prediction = danger (attendu: danger)
```

```
=====
'''
```

ai/predict.py

Type: text | Source: /mnt/e/projects/iot-fire-detection-ai/ai/predict.py

```
FILE: ai/predict.py
'''python
'''
=====
Tâche 3 - Bourti Ayoub
Projet IoT - Système intelligent de détection d'incendie
predict.py : Script de prédiction compatible Node-RED
=====

USAGE :
Mode arguments positionnels (Node-RED exec node) :
python predict.py <temperature> <smoke> <gas> <humidity>
Exemple : python predict.py 78 850 720 22

Mode JSON stdin (API / pipe) :
echo '{"temperature":78,"smoke":850,"gas":720,"humidity":22}' | python predict.py

SORTIE JSON :
{"ai_status": "danger", "risk_score": 0.97, "alert": true}
=====
'''

import sys
import json
import os
import joblib
import numpy as np

# =====
# Chemin du modèle (même dossier que ce script)
# =====
SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
MODEL_PATH = os.path.join(SCRIPT_DIR, "model_fire_detection.joblib")

# =====
# Chargement du modèle (une seule fois)
# =====
def load_model():
    if not os.path.exists(MODEL_PATH):
        error = {"error": f"Modèle introuvable : {MODEL_PATH}. Lancer train_model.py d'abord."}
        print(json.dumps(error))
        sys.exit(1)
    bundle = joblib.load(MODEL_PATH)
    return bundle["model"], bundle["label_encoder"], bundle["features"]

# =====
# Fonction de prédiction principale
# =====
def predict(temperature: float, smoke: float, gas: float, humidity: float) -> dict:
    """
    Prédit le niveau de risque d'incendie.

    Paramètres :
        temperature : float - température en °C
        smoke       : float - indice de fumée
        gas         : float - indice de gaz
        humidity    : float - humidité en %

    Retourne :
        dict avec ai_status (str), risk_score (float), alert (bool)
    """
    model, le, features = load_model()

    X = np.array([[temperature, smoke, gas, humidity]])

    # Classe prédite
    y_enc = model.predict(X)[0]
    ai_status = le.inverse_transform([y_enc])[0]

    # Probabilités par classe
    probas = model.predict_proba(X)[0] # shape (n_classes,)
    classes = le.inverse_transform(model.classes_) # noms des classes encodées

    # risk_score = probabilité de la classe prédite
```



```

from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay,
)
from sklearn.preprocessing import LabelEncoder
import joblib

# #####
# 1. Paramètres
# #####
DATASET_PATH = "dataset_fire_detection.csv"
MODEL_PATH = "model_fire_detection.joblib"
EVAL_PATH = "evaluation_results.txt"
CM_PATH = "confusion_matrix.png"
RANDOM_STATE = 42
TEST_SIZE = 0.25
LABEL_ORDER = ["normal", "suspect", "danger"] # ordre logique

print("=" * 60)
print("  Projet IoT - Détection d'incendie | Tâche 3")
print("  Bourti Ayoub – Entraînement du modèle IA")
print("=" * 60)

# #####
# 2. Chargement et inspection du dataset
# #####
print("\n[1/6] Chargement du dataset ...")
df = pd.read_csv(DATASET_PATH)
print(f"  → {len(df)} échantillons chargés")
print(f"  → Colonnes : {list(df.columns)}")
print(f"  → Distribution des classes :")
for cls in LABEL_ORDER:
    count = (df["label"] == cls).sum()
    print(f"    {cls:<10}: {count} exemples")

# Vérification : pas de valeurs manquantes
assert df.isnull().sum().sum() == 0, "Valeurs manquantes détectées !"
print("  → Aucune valeur manquante détectée ✓")

# #####
# 3. Préparation des données
# #####
print("\n[2/6] Préparation des données ...")
FEATURES = ["temperature", "smoke", "gas", "humidity"]
X = df[FEATURES].values
y = df["label"].values

# Encodage des étiquettes (ordre logique)
le = LabelEncoder()
le.fit(LABEL_ORDER)
y_enc = le.transform(y)
print(f"  → Classes encodées : {dict(zip(le.classes_, le.transform(le.classes_)))}")

X_train, X_test, y_train, y_test = train_test_split(
    X, y_enc, test_size=TEST_SIZE, random_state=RANDOM_STATE, stratify=y_enc
)
print(f"  → Train : {len(X_train)} exemples | Test : {len(X_test)} exemples")

# #####
# 4. Entraînement – Random Forest (modèle principal)
# #####
print("\n[3/6] Entraînement du Random Forest Classifier ...")
rf_model = RandomForestClassifier(
    n_estimators=100,
    max_depth=None,
    min_samples_split=2,
    random_state=RANDOM_STATE,
    class_weight="balanced",
)
rf_model.fit(X_train, y_train)
print("  → Entraînement terminé ✓")

# #####
# 5. Évaluation complète
# #####
print("\n[4/6] Évaluation du modèle ...")

y_pred = rf_model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
report = classification_report(
    y_test, y_pred, target_names=le.classes_
)

```

```
cm = confusion_matrix(y_test, y_pred, labels=le.transform(LABEL_ORDER))

# Cross-validation (5-fold stratifié)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
cv_scores = cross_val_score(rf_model, X, y_enc, cv=cv, scoring="accuracy")

# Importance des features
feat_imp = pd.Series(rf_model.feature_importances_, index=FEATURES).sort_values(ascending=False)

print(f" → Accuracy : {accuracy * 100:.2f} %")
print(f" → CV Score : {cv_scores.mean() * 100:.2f} % ± {cv_scores.std() * 100:.2f} %")

# ████████████████████████████████████████████████████████████
# 6. Sauvegarde des résultats d'évaluation
# ████████████████████████████████████████████████████████████
print("\n[5/6] Sauvegarde des fichiers ...")

# 6a. Texte d'évaluation
eval_text = f"""
=====
EVALUATION DU MODELE IA - Detection d incendie
Tache 3 - Bourti Ayoub
=====

Modele      : Random Forest Classifier
Dataset     : {DATASET_PATH}
Echantillons : {len(df)} total | {len(X_train)} train | {len(X_test)} test
Features    : {' , '.join(FEATURES)}
Classes     : {' , '.join(LABEL_ORDER)}

-----
ACCURACY SUR LE JEU DE TEST
-----
Accuracy    : {accuracy * 100:.2f} %

-----
CROSS-VALIDATION (5-Fold Stratifiée)
-----
Scores      : {[f"{s*100:.2f}" for s in cv_scores]}
Moyenne     : {cv_scores.mean() * 100:.2f} %
Ecart-type  : {cv_scores.std() * 100:.2f} %

-----
RAPPORT DE CLASSIFICATION
-----
{report}

-----
MATRICE DE CONFUSION (lignes=reel, colonnes=predit)
-----
Classes     : {' , '.join(LABEL_ORDER)}
{cm}

-----
IMPORTANCE DES FEATURES
-----
{feat_imp.to_string()}

-----
TESTS SUR LES 3 SCENARIOS DU PROJET
-----
Scenario 1 - Normal : temp=24 smoke=80 gas=120 humid=55
Prediction = {le.inverse_transform(rf_model.predict([[24, 80, 120, 55]]))[0]} (attendu: normal)

Scenario 2 - Suspect : temp=48 smoke=340 gas=300 humid=40
Prediction = {le.inverse_transform(rf_model.predict([[48, 340, 300, 40]]))[0]} (attendu: suspect)

Scenario 3 - Danger : temp=78 smoke=850 gas=720 humid=22
Prediction = {le.inverse_transform(rf_model.predict([[78, 850, 720, 22]]))[0]} (attendu: danger)

=====
"""""

with open(EVAL_PATH, "w", encoding="utf-8") as f:
    f.write(eval_text)
print(f" → {EVAL_PATH} sauvegardé ✓")

# 6b. Confusion matrix PNG
fig, ax = plt.subplots(figsize=(7, 5))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="YlOrRd",
    xticklabels=LABEL_ORDER,
    yticklabels=LABEL_ORDER,
    linewidths=0.5,
    ax=ax,
```


weighted avg	1.00	1.00	1.00	30
--------------	------	------	------	----

MATRICE DE CONFUSION (lignes=reel, colonnes=predit)

Classes : normal, suspect, danger
[[10 0 0]
[0 10 0]
[0 0 10]]

IMPORTANCE DES FEATURES

smoke	0.310065
temperature	0.282858
gas	0.277583
humidity	0.129495

TESTS SUR LES 3 SCENARIOS DU PROJET

Scenario 1 - Normal : temp=24 smoke=80 gas=120 humid=55
Prediction = normal (attendu: normal)

Scenario 2 - Suspect : temp=48 smoke=340 gas=300 humid=40
Prediction = suspect (attendu: suspect)

Scenario 3 - Danger : temp=78 smoke=850 gas=720 humid=22
Prediction = danger (attendu: danger)

=====
```\n

## cloud/preuve\_test\_bout\_en\_bout.md

Type: text | Source: /mnt/e/projects/iot-fire-detection-ai/cloud/preuve\_test\_bout\_en\_bout.md

FILE: cloud/preuve\_test\_bout\_en\_bout.md  
```markdown

Preuve de validation : test bout en bout

Responsable : LOUAHABI Abdenour

Composante : Tâche 5 (simulation edge et finalisation de la pipeline cloud)

Objectif du test

Ce document valide le fonctionnement global de la chaîne IoT et IA. Le test vise à montrer que les données environnementales

Architecture validée

Le chemin de traitement validé est le suivant :

1. **Génération edge :** le script Python génère des valeurs réalistes de température, fumée, gaz et humidité.
2. **Publication MQTT :** le script Python publie les données vers le broker MQTT configuré. Pour la démonstration locale,
3. **Orchestration et IA :** Node-RED reçoit le payload, appelle `predict.py`, puis enrichit le JSON avec `ai\_status`, `ris
4. **Visualisation cloud :** ThingsBoard reçoit la télémétrie finale via MQTT sur `v1/devices/me/telemetry` et met à jour l

Preuves visuelles

Les captures d'écran disponibles dans `screenshots/` documentent les trois scénarios du système :

- `dashboard\_normal.png` : statut IA `normal`, alerte `false`
- `dashboard\_suspect.png` : statut IA `suspect`, alerte `false`
- `dashboard\_danger.png` : statut IA `danger`, alerte `true`

Conclusion

Le flux de données de bout en bout est cohérent pour une démonstration académique. La communication entre composants, l'enr

```\n

## cloud/README\_demo.md

Type: text | Source: /mnt/e/projects/iot-fire-detection-ai/cloud/README\_demo.md

FILE: cloud/README\_demo.md  
```markdown

Démonstration : Système intelligent de détection d'incendie

Ce document décrit la procédure de démonstration locale du système IoT de détection d'incendie, depuis le simulateur Python

Prérequis

- Python 3.x installé
- dépendances Python installées avec `pip install -r requirements.txt`
- Mosquitto installé et disponible localement
- Node-RED installé et accessible sur `http://localhost:1880`
- flow `integration\_ia\_node\_red/flow\_node\_red\_ai\_alert.json` importé et déployé
- tableau de bord ThingsBoard configuré avec votre propre device token

Configuration MQTT locale

La démonstration locale utilise par défaut :

- broker MQTT : `localhost:1883`
- topic capteurs : `iot/fire/sensor/data`
- topic alertes : `iot/fire/alert`

Un broker public peut être utilisé à titre optionnel pour des tests distants, mais il n'est pas requis pour la démonstration

Étapes de lancement

1. Démarrer Mosquitto

```
```bash
sudo service mosquitto start
```
```

2. Démarrer Node-RED

```
```bash
node-red
```
```

3. Déployer le flow Node-RED

Importer `integration\_ia\_node\_red/flow\_node\_red\_ai\_alert.json`, vérifier le broker local, puis déployer le flow.

4. Démarrer le simulateur

Depuis la racine du projet :

```
```bash
python simulator/simulate_fire_sensors.py
```
```

Le script publie les données vers le broker MQTT configuré. Pour la démonstration locale, le broker utilisé est `localhost:1883`

5. Vérifier la réception MQTT

```
```bash
mosquitto_sub -h localhost -t iot/fire/sensor/data
```
```

6. Vérifier l'orchestration et le cloud

- observer la sortie debug dans Node-RED
- vérifier l'enrichissement avec `pre\_alert`, `ai\_status`, `risk\_score` et `alert`
- confirmer l'affichage des données sur le tableau de bord ThingsBoard

...

docs/checklist_soutenance.md

Type: text | Source: /mnt/e/projects/iot-fire-detection-ai/docs/checklist_soutenance.md

FILE: docs/checklist_soutenance.md

```
```markdown
```

# Checklist de soutenance

## ## Démonstration

- Démarrer Mosquitto
- Démarrer Node-RED
- Importer et déployer le flow Node-RED
- Démarrer le simulateur
- Montrer la réception MQTT
- Montrer la sortie debug Node-RED
- Montrer la prédiction IA
- Montrer l'alerte danger
- Montrer le dashboard ThingsBoard

## ## Problèmes fréquents et corrections

- `paho-mqtt` non installé : exécuter `pip install -r requirements.txt`
- Mosquitto non démarré : exécuter `sudo service mosquitto start`
- chemin `predict.py` incorrect dans Node-RED : ajuster la commande du node `Call predict.py`
- token ThingsBoard absent : renseigner le token du device
- topic MQTT incohérent : vérifier `iot/fire/sensor/data` et `iot/fire/alert`

...

## docs/mqtt\_format\_membre1.md

Type: text | Source: /mnt/e/projects/iot-fire-detection-ai/docs/mqtt\_format\_membre1.md

```
FILE: docs/mqtt_format_membre1.md
```markdown
# Membre 1 - Simulation des capteurs et publication MQTT

## Rôle technique

Cette partie représente la couche objets / edge du projet IoT. Elle simule les capteurs d'un système intelligent de détection de feu.

## Capteurs simulés

| Capteur | Description |
|---|---|
| `temperature` | Température ambiante en degres Celsius |
| `smoke` | Indice simule de fumee |
| `gas` | Indice simule de gaz |
| `humidity` | Humidite de l'air en pourcentage |

## Configuration MQTT standard

- broker local par default : `localhost:1883`
- topic capteurs : `iot/fire/sensor/data`
- topic alertes : `iot/fire/alert`

Des brokers publics peuvent etre utilises a titre optionnel pour des essais distants, mais ils ne sont pas requis pour la détection de feu.

## Format JSON commun

```json
{
 "device_id": "fire_sensor_01",
 "timestamp": "2026-05-26T10:30:00",
 "temperature": 35.2,
 "smoke": 180,
 "gas": 220,
 "humidity": 45,
 "location": "Salle_1",
 "scenario": "normal"
}
```

## Scénarios simulés

### Normal

- Température : 20 à 35 °C
- Fumée : 20 à 200
- Gaz : 50 à 250
- Humidité : 45 à 65 %

### Suspect

- Température : 40 à 60 °C
- Fumée : 250 à 600
- Gaz : 300 à 550
- Humidité : 30 à 45 %

### Danger

- Température : 65 à 90 °C
- Fumée : 650 à 1000
- Gaz : 600 à 900
- Humidité : 15 à 30 %

## Ce que le Membre 2 utilisera

Le Membre 2 doit configurer Node-RED avec :

```text
MQTT IN topic = iot/fire/sensor/data
MQTT OUT topic = iot/fire/alert
Broker local = localhost:1883
```
```

Puis parser le payload JSON reçu.

...

docs/README_node_red_base(membre2).md

Type: text | Source: /mnt/e/projects/iot-fire-detection-ai/docs/README_node_red_base(membre2).md

```
FILE: docs/README_node_red_base(membre2).md
```markdown
README - Node-RED Base Flow

Projet IoT & IA – Système intelligent de détection d’incendie

1. Objectif de la tâche

Cette tâche consiste à construire la première couche de traitement Node-RED du système IoT de détection intelligente d’incendie.

L’objectif principal est de :

* recevoir les données capteurs via MQTT,
* vérifier la validité des messages JSON,
* normaliser les valeurs numériques,
* appliquer une première logique de pré-détection de risque,
* produire un payload propre et exploitable pour la partie Intelligence Artificielle.

Cette étape joue un rôle central dans le pipeline du projet, car elle garantit que les données transmises au modèle IA sont valides et exploitables.

2. Architecture du flow réalisé

Le flow Node-RED développé suit l’architecture suivante :


```

graph TD
    MQTT[MQTT Broker] --> Input[MQTT Input Node]
    Input --> Validation[Payload Validation]
    Validation --> Normalization[Data Normalization]
    Normalization --> Logic[Pre-Alert Logic]
    Logic --> Output[Debug Output]

```

3. Technologies utilisées

| Technologie | Rôle |
|-------------|---|
| Node-RED | Orchestration et traitement des données |
| MQTT | Communication publish/subscribe |
| Mosquitto | Broker MQTT local |
| Python | Simulation des capteurs incendie |
| JSON | Format des messages échangés |

4. Topic MQTT utilisé


```

iot/fire/sensor/data

```


Le flow est abonné à ce topic afin de recevoir les données publiées par le simulateur Python.

5. Structure JSON traitée

Le flow traite les messages JSON suivants :


```

{
  "device_id": "fire_sensor_01",
  "timestamp": "2026-05-26T23:41:52",
  "temperature": 76.8,
  "smoke": 904,

```


```

```
 "gas": 731,
 "humidity": 25,
 "location": "Salle_1",
 "scenario": "danger"
 },
 ...
}
```

---

## # 6. Fonctionnalités implémentées

### ## 6.1 Réception MQTT

Le node `MQTT Input` reçoit les données envoyées en temps réel par le simulateur de capteurs incendie.

---

### ## 6.2 Validation du payload

Le node `Validate Sensor Payload` vérifie :

- \* la présence des champs obligatoires,
- \* la cohérence minimale des données,
- \* la validité des valeurs numériques.

Les messages invalides sont automatiquement ignorés.

---

### ## 6.3 Normalisation des données

Le node `Normalize Sensor Values` convertit les valeurs reçues en types numériques afin d'éviter les erreurs de traitement.

Les champs normalisés sont :

- \* temperature
- \* smoke
- \* gas
- \* humidity

---

### ## 6.4 Logique de pré-alerte

Le node `Fire Risk Pre-Alert Logic` applique une première logique simple basée sur des seuils.

Trois niveaux de risque sont générés :

Niveau	Description
normal	Situation normale
suspect	Valeurs inhabituelles
danger	Risque élevé d'incendie

Le résultat est ajouté dans le champ :

```
````json  
"pre_alert"  
````
```

Exemple :

```
````json  
{  
  "pre_alert": "danger"  
}  
````
```

---

## # 7. Exemple de payload final

```
````json  
{  
  "device_id": "fire_sensor_01",  
  "timestamp": "2026-05-26T23:41:52",  
  "temperature": 76.8,  
  "smoke": 904,  
  "gas": 731,  
  "humidity": 25,  
  "location": "Salle_1",  
  "scenario": "danger",  
  "pre_alert": "danger"  
}  
````
```

---

```

8. Exécution du projet

Étape 1 – Démarrer Mosquitto

```bash
mosquitto
```

Étape 2 – Démarrer Node-RED

```bash
node-red
```

Puis ouvrir :

```text
http://127.0.0.1:1880
```

Étape 3 – Lancer le simulateur Python

```bash
python simulate_fire_sensors.py
```

Étape 4 – Déployer le flow

Cliquer sur le bouton :

```text
Deploy
```

Étape 5 – Observer les résultats

Les payloads propres et enrichis sont visibles dans le panneau Debug de Node-RED.

9. Livrables produits

Fichier	Description
flow_node_red_base.json	Flow Node-RED exporté
README_node_red_base.md	Documentation technique
capture_node_red_reception.png	Réception MQTT
capture_debug_payload_clean.png	Payload propre après traitement

10. Résultat obtenu

Le flow Node-RED fonctionne correctement et permet :

- la réception des données MQTT,
- le traitement des payloads JSON,
- la validation des données,
- la normalisation des valeurs,
- la génération d'un niveau de pré-alerte.

Le système est maintenant prêt pour l'intégration de la partie Intelligence Artificielle dans la tâche suivante.

```

## docs/README\_task3.md

Type: text | Source: /mnt/e/projects/iot-fire-detection-ai/docs/README\_task3.md

```

FILE: docs/README_task3.md
```markdown
# Tâche 3 – Dataset IA, Entraînement et Prédiction Python
**Projet IoT - Système intelligent de détection d'incendie**
**Responsable : Bourti Ayoub**

```

```

---

## Contenu du dossier `ai/`

| Fichier | Rôle |
|---|---|
| `dataset_fire_detection.csv` | Dataset équilibré (120 lignes, 3 classes) |
| `train_model.py` | Script d'entraînement, évaluation et sauvegarde |
| `predict.py` | Script de prédiction compatible Node-RED |
| `model_fire_detection.joblib` | Modèle sauvegardé (généré par `train_model.py`) |
| `evaluation_results.txt` | Métriques complètes : accuracy, CV, rapport, scénarios |
| `confusion_matrix.png` | Matrice de confusion visuelle |

---

## Prérequis

```bash
pip install scikit-learn pandas numpy matplotlib seaborn joblib
```

Python 3.8+ recommandé.

---

## Étape 1 – Entraîner le modèle

```bash
python train_model.py
```

Ce script :
1. Charge `dataset_fire_detection.csv`
2. Divise les données (75% train / 25% test)
3. Entraîne un **Random Forest Classifier** (100 arbres)
4. Évalue avec accuracy, cross-validation 5-fold et rapport de classification
5. Génère `model_fire_detection.joblib`, `evaluation_results.txt`, `confusion_matrix.png`

Résultat attendu :
```
Accuracy : 100.00 %
CV Moyenne : 100.00 %
[✓] Normal → normal
[✓] Suspect → suspect
[✓] Danger → danger
```

---

## Étape 2 – Utiliser predict.py

### Mode 1 : Arguments positionnels (Node-RED `exec` node)

```bash
python predict.py <temperature> <smoke> <gas> <humidity>
```

Exemples des 3 scénarios :

```bash
Scénario normal
python predict.py 24 80 120 55
→ {"ai_status": "normal", "risk_score": 1.0, "alert": false}

Scénario suspect
python predict.py 48 340 300 40
→ {"ai_status": "suspect", "risk_score": 1.0, "alert": false}

Scénario danger
python predict.py 78 850 720 22
→ {"ai_status": "danger", "risk_score": 1.0, "alert": true}
```

### Mode 2 : JSON via stdin (pipe / API)

```bash
echo '{"temperature":78,"smoke":850,"gas":720,"humidity":22}' | python predict.py
→ {"ai_status": "danger", "risk_score": 1.0, "alert": true}
```

---

## Sortie JSON de predict.py

```json
{
 "ai_status": "danger",

```

```

 "risk_score": 0.97,
 "alert": true
}
```

Champ	Type	Description
`ai_status`	string	Classe prédite : `normal`, `suspect` ou `danger`
`risk_score`	float	Probabilité de la classe prédite (0.0 → 1.0)
`alert`	boolean	`true` uniquement si `ai_status == "danger"`

---

## Intégration dans Node-RED (pour la Tâche 4)

Dans Node-RED, ajouter un exec` node avec la commande :

```
python /chemin/vers/ai/predict.py {{{payload.temperature}}} {{{payload.smoke}}} {{{payload.gas}}} {{{payload.humidity}}}
```

Puis parser la sortie avec un json` node pour obtenir ai_status, risk_score et alert directement dans msg.payload

---

## Description du dataset

Le dataset contient 120 exemples équilibrés (40 par classe) avec les colonnes :

- temperature` (°C)
- smoke` (indice 0-1000)
- gas` (indice 0-850)
- humidity` (%)
- label` : normal` | suspect` | danger`

#### Plages de valeurs par classe

Classe	Température	Fumée	Gaz	Humidité
normal`	16-35°C	30-115	60-145	45-65%
suspect`	41-60°C	230-510	250-450	31-47%
danger`	63-100°C	700-1000	580-850	10-29%

Ces plages sont cohérentes avec celles définies dans le contrat commun du projet.

---

## Modèle choisi : Random Forest Classifier

- Pourquoi ? Robuste sur données tabulaires, résistant au surapprentissage, interprétable via l'importance des features
- Hyperparamètres : 100 arbres, class_weight="balanced", random_state=42`
- Alternative testable : DecisionTreeClassifier` pour visualisation graphique de l'arbre

---

## Livrables transmis à la Tâche 4

Livrable	Utilisé par
predict.py`	Node-RED via exec` node
model_fire_detection.joblib`	Chargé par predict.py`
evaluation_results.txt`	Documentation / rapport final
confusion_matrix.png`	Rapport final / soutenance

---

*Projet encadré par : Pr. M. EL Brak – FST Tanger – Année universitaire 2025/2026*

```

docs/validation_membre1.md

Type: text | Source: /mnt/e/projects/iot-fire-detection-ai/docs/validation_membre1.md

```

FILE: docs/validation_membre1.md
```markdown
Validation du travail - Membre 1

Commandes de test

Test hors ligne rapide

```bash
python simulator/simulate_fire_sensors.py --offline --once
```

```

Résultat attendu : un seul message JSON généré.

### Test continu

```
```bash
python simulator/simulate_fire_sensors.py
```
```

Résultat attendu :

- si MQTT est accessible : `Message publié` ;
- si MQTT est inaccessible : `Message généré (mode hors ligne)`.

## Checklist

- [ ] Le script démarre sans erreur.
- [ ] Le fichier `config.json` est lu correctement.
- [ ] Le topic est `iot/fire/sensor/data`.
- [ ] Un message JSON est généré toutes les 3 secondes.
- [ ] Le JSON contient `device\_id`, `timestamp`, `temperature`, `smoke`, `gas`, `humidity`, `location`, `scenario`.
- [ ] Les scénarios `normal`, `suspect` et `danger` sont présents.
- [ ] Le script ne s'arrête pas brutalement si Internet ou le broker public est inaccessible.

## Validation pour passer au Membre 2

Le Membre 2 peut commencer lorsque le Membre 1 fournit :

```
```text
simulator/simulate_fire_sensors.py
simulator/config.json
docs/mqtt_format_membre1.md
```
```

## flow/flow\_node\_red\_base.json

Type: text | Source: /mnt/e/projects/iot-fire-detection-ai/flow/flow\_node\_red\_base.json

FILE: flow/flow\_node\_red\_base.json  
```json

```
[
  {
    "id": "5bb61ebb48387936",
    "type": "tab",
    "label": "project flow",
    "disabled": false,
    "info": "",
    "env": []
  },
  {
    "id": "cb0e1123fc348e2c",
    "type": "mqtt in",
    "z": "5bb61ebb48387936",
    "name": "",
    "topic": "iot/fire/sensor/data",
    "qos": "2",
    "datatype": "auto-detect",
    "broker": "d2895a1b9b0fc846",
    "nl": false,
    "rap": true,
    "rh": 0,
    "inputs": 0,
    "x": 390,
    "y": 440,
    "wires": [
      [
        "a23071e09ec7ab36"
      ]
    ]
  },
  {
    "id": "a23071e09ec7ab36",
    "type": "function",
    "z": "5bb61ebb48387936",
    "name": "Validate Sensor Payload",
    "func": "let p = msg.payload;\n\nconst requiredFields = [\\n    \\\"device_id\\\",\\n    \\\"timestamp\\\",\\n    \\\"temperature\\",\\n    \\\"smoke\\\",\\n    \\\"gas\\\",\\n    \\\"humidity\\\",\\n    \\\"location\\\",\\n    \\\"scenario\\\";\n\nif (p === null || p === undefined) {\n  msg.error = \"Payload is null or undefined\";\n  return [msg];\n}\n\nfor (let i = 0; i < requiredFields.length; i++) {\n  if (p[requiredFields[i]] === null || p[requiredFields[i]] === undefined) {\n    msg.error = \"Missing required field: \" + requiredFields[i];\n    return [msg];\n  }\n}\n\nmsg.valid = true;\nreturn [msg];",
    "outputs": 1,
    "timeout": 0,
    "noerr": 0,
    "initialize": "",
    "finalize": "",
    "libs": [],
    "x": 690,
```

```

        "y": 440,
        "wires": [
            [
                "ef5bfe5528a1fbc6"
            ]
        ]
    },
    {
        "id": "ef5bfe5528a1fbc6",
        "type": "function",
        "z": "5bb61ebb48387936",
        "name": "Normalize Sensor Values",
        "func": "let p = msg.payload;\n\np.temperature = Number(p.temperature);\np.smoke = Number(p.smoke);\np.gas = Number(p.gas);\nreturn p",
        "outputs": 1,
        "timeout": 0,
        "noerr": 0,
        "initialize": "",
        "finalize": "",
        "libs": [],
        "x": 950,
        "y": 440,
        "wires": [
            [
                "551d5cf66cbb8629"
            ]
        ]
    },
    {
        "id": "551d5cf66cbb8629",
        "type": "function",
        "z": "5bb61ebb48387936",
        "name": "Fire Risk Pre-Alert Logic",
        "func": "let p = msg.payload;\n\nif (\n    p.temperature > 65 ||\n    p.smoke > 650 ||\n    p.gas > 600\n) {\n    return p\n}",
        "outputs": 1,
        "timeout": 0,
        "noerr": 0,
        "initialize": "",
        "finalize": "",
        "libs": [],
        "x": 1190,
        "y": 440,
        "wires": [
            [
                "8228209590984eee"
            ]
        ]
    },
    {
        "id": "8228209590984eee",
        "type": "debug",
        "z": "5bb61ebb48387936",
        "name": "Clean Payload Output",
        "active": true,
        "tosidebar": true,
        "console": false,
        "tostatus": false,
        "complete": "payload",
        "targetType": "msg",
        "statusVal": "",
        "statusType": "auto",
        "x": 1440,
        "y": 440,
        "wires": []
    },
    {
        "id": "d2895a1b9b0fc846",
        "type": "mqtt-broker",
        "name": "Local Mosquitto",
        "broker": "localhost",
        "port": 1883,
        "clientId": "",
        "autoConnect": true,
        "usetls": false,
        "protocolVersion": 4,
        "keepalive": 60,
        "cleansession": true,
        "autoUnsubscribe": true,
        "birthTopic": "",
        "birthQos": "0",
        "birthRetain": "false",
        "birthPayload": "",
        "birthMsg": {},
        "closeTopic": "",
        "closeQos": "0",
        "closeRetain": "false",
        "closePayload": "",
        "closeMsg": {}
    }

```

```

    "willTopic": "",
    "willQos": "0",
    "willRetain": "false",
    "willPayload": "",
    "willMsg": {},
    "userProps": "",
    "sessionExpiry": ""
  }
},
],

```

integration_ia_node_red/flow_node_red_ai_alert.json

Type: text | Source: /mnt/e/projects/iot-fire-detection-ai/integration_ia_node_red/flow_node_red_ai_alert.json

```

FILE: integration_ia_node_red/flow_node_red_ai_alert.json
```json
[
 {
 "id": "3f9af7491622b8c9",
 "type": "tab",
 "label": "project flow",
 "disabled": false,
 "info": "",
 "env": []
 },
 {
 "id": "db631d80f455a1a1",
 "type": "mqtt in",
 "z": "3f9af7491622b8c9",
 "name": "",
 "topic": "iot/fire/sensor/data",
 "qos": "2",
 "datatype": "auto-detect",
 "broker": "d2895a1b9b0fc846",
 "nl": false,
 "rap": true,
 "rh": 0,
 "inputs": 0,
 "x": 390,
 "y": 440,
 "wires": [
 [
 "2e3c72aa901e77cd"
]
]
 },
 {
 "id": "2e3c72aa901e77cd",
 "type": "function",
 "z": "3f9af7491622b8c9",
 "name": "Validate Sensor Payload",
 "func": "let p = msg.payload;\n\nconst requiredFields = [\n \"device_id\", \n \"timestamp\", \n \"temperature\"",
 "outputs": 1,
 "timeout": 0,
 "noerr": 0,
 "initialize": "",
 "finalize": "",
 "libs": [],
 "x": 550,
 "y": 540,
 "wires": [
 [
 "7f41a188c45d1b7b"
]
]
 },
 {
 "id": "7f41a188c45d1b7b",
 "type": "function",
 "z": "3f9af7491622b8c9",
 "name": "Normalize Sensor Values",
 "func": "let p = msg.payload;\n\np.temperature = Number(p.temperature);\n\np.smoke = Number(p.smoke);\n\np.gas = Number(p.gas);",
 "outputs": 1,
 "timeout": 0,
 "noerr": 0,
 "initialize": "",
 "finalize": "",
 "libs": [],
 "x": 690,
 "y": 380,
 "wires": [
 [
 "e78e97850e326c35"
]
]
 }
]

```

```

]
],
 {
 "id": "e78e97850e326c35",
 "type": "function",
 "z": "3f9af7491622b8c9",
 "name": "Fire Risk Pre-Alert Logic",
 "func": "let p = msg.payload;\n\nif (\n p.temperature > 65 ||\n p.smoke > 650 ||\n p.gas > 600\n) {\n p
 "outputs": 1,
 "timeout": 0,
 "noerr": 0,
 "initialize": "",
 "finalize": "",
 "libs": [],
 "x": 890,
 "y": 540,
 "wires": [
 [
 "beec55db842a12b9"
]
]
 },
 {
 "id": "db1ae0a677fd18a3",
 "type": "debug",
 "z": "3f9af7491622b8c9",
 "name": "Clean Payload Output",
 "active": true,
 "tosidebar": true,
 "console": false,
 "tostatus": false,
 "complete": "payload",
 "targetType": "msg",
 "statusVal": "",
 "statusType": "auto",
 "x": 1660,
 "y": 440,
 "wires": []
 },
 {
 "id": "beec55db842a12b9",
 "type": "function",
 "z": "3f9af7491622b8c9",
 "name": "Prepare IA Command",
 "func": "let p = msg.payload;\n\n// Garder une copie du message capteur original.\n// Le node exec va remplacer msg
 "outputs": 1,
 "timeout": 0,
 "noerr": 0,
 "initialize": "",
 "finalize": "",
 "libs": [],
 "x": 1000,
 "y": 380,
 "wires": [
 [
 "c2747d576065d6bf"
]
]
 },
 {
 "id": "c2747d576065d6bf",
 "type": "exec",
 "z": "3f9af7491622b8c9",
 "command": "python ai/predict.py",
 "addpay": "payload",
 "append": "",
 "useSpawn": "false",
 "timer": "",
 "winHide": false,
 "oldrc": false,
 "name": "Call predict.py",
 "x": 1200,
 "y": 540,
 "wires": [
 [
 "cdb3046ebe4311fb"
],
 [],
 []
]
 },
 {
 "id": "cdb3046ebe4311fb",
 "type": "json",
 "z": "3f9af7491622b8c9",
 "name": "Parse IA JSON",

```

```

 "property": "payload",
 "action": "",
 "pretty": false,
 "x": 1300,
 "y": 460,
 "wires": [
 [
 "b978f11506a54a89"
]
]
 },
 {
 "id": "b978f11506a54a89",
 "type": "function",
 "z": "3f9af7491622b8c9",
 "name": "Merge IA Result",
 "func": "let sensor = msg.sensor;\nlet ai = msg.payload;\n\n// Vérification : est-ce qu'on a encore le message captu",
 "outputs": 1,
 "timeout": 0,
 "noerr": 0,
 "initialize": "",
 "finalize": "",
 "libs": [],
 "x": 1400,
 "y": 360,
 "wires": [
 [
 "db1ae0a677fd18a3",
 "f3a4503e6943fc47"
]
]
 },
 {
 "id": "f3a4503e6943fc47",
 "type": "switch",
 "z": "3f9af7491622b8c9",
 "name": "Is Danger?",
 "property": "payload.alert",
 "propertyType": "msg",
 "rules": [
 {
 "t": "true"
 }
],
 "checkall": "true",
 "repair": false,
 "outputs": 1,
 "x": 1630,
 "y": 320,
 "wires": [
 [
 "2eece650a6563049",
 "876846355d647fe1"
]
]
 },
 {
 "id": "2eece650a6563049",
 "type": "debug",
 "z": "3f9af7491622b8c9",
 "name": "ALERTE DANGER",
 "active": true,
 "tosidebar": true,
 "console": false,
 "tostatus": false,
 "complete": "payload",
 "targetType": "msg",
 "statusVal": "",
 "statusType": "auto",
 "x": 1810,
 "y": 280,
 "wires": []
 },
 {
 "id": "876846355d647fe1",
 "type": "mqtt out",
 "z": "3f9af7491622b8c9",
 "name": "Publish Alert MQTT",
 "topic": "iot/fire/alert",
 "qos": "",
 "retain": "",
 "respTopic": "",
 "contentType": "",
 "userProps": "",
 "correl": "",
 "expiry": "",
 "broker": "d2895a1b9b0fc846",
 }
}

```

```

 "x": 1850,
 "y": 340,
 "wires": []
 },
 {
 "id": "d2895a1b9b0fc846",
 "type": "mqtt-broker",
 "name": "Local Mosquitto",
 "broker": "localhost",
 "port": 1883,
 "clientId": "",
 "autoConnect": true,
 "usetls": false,
 "protocolVersion": 4,
 "keepalive": 60,
 "cleansession": true,
 "autoUnsubscribe": true,
 "birthTopic": "",
 "birthQos": "0",
 "birthRetain": "false",
 "birthPayload": "",
 "birthMsg": {},
 "closeTopic": "",
 "closeQos": "0",
 "closeRetain": "false",
 "closePayload": "",
 "closeMsg": {},
 "willTopic": "",
 "willQos": "0",
 "willRetain": "false",
 "willPayload": "",
 "willMsg": {},
 "userProps": "",
 "sessionExpiry": ""
 }
]
...

```

## integration\_ia\_node\_red/README\_integration\_ia.md

Type: text | Source: /mnt/e/projects/iot-fire-detection-ai/integration\_ia\_node\_red/README\_integration\_ia.md

```

FILE: integration_ia_node_red/README_integration_ia.md
```markdown
# Tâche 4 - Intégration IA dans Node-RED et logique d'alerte

## 1. Objectif

L'objectif de cette tâche est d'intégrer le modèle d'intelligence artificielle dans le flow Node-RED afin de classifier aut
Node-RED reçoit les données des capteurs via MQTT, appelle le script Python `predict.py`, récupère la prédiction IA, enrichi
---

## 2. Entrées utilisées

Cette tâche utilise les fichiers suivants :

- `flow_node_red_base.json` : flow Node-RED de base fourni par la tâche 2.
- `predict.py` : script Python de prédiction fourni par la tâche 3.
- `model_fire_detection.joblib` : modèle IA entraîné fourni par la tâche 3.
- Messages MQTT publiés par le simulateur sur le topic :

```text
iot/fire/sensor/data
```
---

## 3. Format du message reçu

Le message reçu par Node-RED respecte le format JSON commun du projet :

```json
{
 "device_id": "fire_sensor_01",
 "timestamp": "2026-05-27T17:31:05",
 "temperature": 32.91,
 "smoke": 115,
 "gas": 172,
 "humidity": 62,
 "location": "Salle_1",
 "scenario": "normal"
}

```

```
}
...
```

Les variables utilisées par l'IA sont :

```
- `temperature`
- `smoke`
- `gas`
- `humidity`
```

```

```

#### ## 4. Fonctionnement du flow Node-RED

Le flow final suit cette chaîne de traitement :

```
```text  
MQTT in  
→ Validate Sensor Payload  
→ Normalize Sensor Values  
→ Fire Risk Pre-Alert Logic  
→ Prepare IA Command  
→ Call predict.py  
→ Parse IA JSON  
→ Merge IA Result  
→ Is Danger?  
→ Publish Alert MQTT  
```
```

```

```

#### ## 5. Préparation de la commande IA

Le node `Prepare IA Command` prépare les quatre valeurs nécessaires pour le script `predict.py`.

Exemple :

```
```text  
24 80 120 55  
```
```

Ces valeurs correspondent à :

```
```text  
temperature smoke gas humidity  
```
```

Le message capteur original est sauvegardé dans `msg.sensor` afin de pouvoir être fusionné ensuite avec le résultat IA.

```

```

#### ## 6. Appel du script Python

Le node `Call predict.py` utilise un node `exec` pour exécuter le script Python suivant :

```
```text  
python ai/predict.py  
```
```

L'option `Joindre msg.payload` est activée afin d'ajouter automatiquement les valeurs des capteurs à la commande.

Exemple de commande exécutée :

```
```text  
python ai/predict.py 78 850 720 22  
```
```

#### ## Important : configuration du chemin predict.py

- Si Node-RED est lancé depuis la racine du projet, utiliser : `python ai/predict.py`
- Si Node-RED est lancé depuis un autre dossier, définir manuellement un chemin absolu.
- Exemple Linux/WSL : `python /mnt/e/projects/iot-fire-detection-ai/ai/predict.py`
- Exemple Windows : `python C:\path\to\iot-fire-detection-ai\ai\predict.py`

```

```

#### ## 7. Résultat retourné par l'IA

Le script `predict.py` retourne un résultat JSON contenant :

```
```json  
{  
  "ai_status": "danger",  
  "risk_score": 1.0,  
  "alert": true  
}
```

Le node `Parse IA JSON` transforme cette réponse texte en objet JSON exploitable par Node-RED.

8. Enrichissement du message

Le node `Merge IA Result` fusionne le message original avec la prédiction IA.

Exemple de message enrichi :

```
```json
{
 "device_id": "fire_sensor_01",
 "timestamp": "2026-05-27T17:31:41",
 "temperature": 41.73,
 "smoke": 576,
 "gas": 324,
 "humidity": 41,
 "location": "Salle_1",
 "scenario": "suspect",
 "pre_alert": "suspect",
 "ai_status": "suspect",
 "risk_score": 1,
 "alert": false
}
```
```

9. Logique d'alerte

La règle d'alerte utilisée est :

```
```text
Si ai_status == "danger"
alors alert = true
sinon alert = false
```
```

Le node `Is Danger?` vérifie la valeur :

```
```text
msg.payload.alert
```
```

Si cette valeur est `true`, le message est envoyé vers :

- le debug `ALERTE DANGER`
- le node MQTT out `Publish Alert MQTT`

10. Publication de l'alerte MQTT

Lorsqu'un danger est détecté, l'alerte est publiée sur le topic MQTT :

```
```text
iot/fire/alert
```
```

Le node utilisé est :

```
```text
Publish Alert MQTT
```
```

Le broker utilisé est :

```
```text
Local Mosquitto
```
```

11. Tests réalisés

Test normal

Commande testée :

```
```text
python ai\predict.py 24 80 120 55
```
```

Résultat obtenu :

```
```json
```

```
{
 "ai_status": "normal",
 "risk_score": 1.0,
 "alert": false
},

```

### Test suspect

Commande testée :

```
```text
python ai\predict.py 48 340 300 40
```
```

Résultat obtenu :

```
```json
{
  "ai_status": "suspect",
  "risk_score": 1.0,
  "alert": false
}
```
```

### Test danger

Commande testée :

```
```text
python ai\predict.py 78 850 720 22
```
```

Résultat obtenu :

```
```json
{
  "ai_status": "danger",
  "risk_score": 1.0,
  "alert": true
}
```
```

---

## 12. Résultats observés dans Node-RED

Les messages MQTT sont bien reçus depuis le simulateur sur le topic :

```
```text
iot/fire/sensor/data
```
```

Node-RED enrichit correctement les messages avec les champs suivants :

- `pre\_alert`
- `ai\_status`
- `risk\_score`
- `alert`

Exemple normal observé :

```
```text
scenario: "normal"
ai_status: "normal"
risk_score: 1
alert: false
```
```

Exemple suspect observé :

```
```text
scenario: "suspect"
ai_status: "suspect"
risk_score: 1
alert: false
```
```

Exemple danger attendu :

```
```text
scenario: "danger"
ai_status: "danger"
risk_score: 1
alert: true
```
```

---

#### ## 13. Problème rencontré et correction

Au début du test MQTT, le simulateur fonctionnait en mode hors ligne avec le message suivant :

```
```text
paho-mqtt n'est pas installé : passage en mode hors ligne.
```
```

Cela signifiait que les messages étaient générés, mais non publiés vers MQTT.

La correction a été d'installer la bibliothèque `paho-mqtt` :

```
```text
pip install paho-mqtt
```
```

La vérification a ensuite été faite avec :

```
```text
python -c "import paho.mqtt.client as mqtt; print('paho-mqtt OK')"
```
```

Résultat :

```
```text
paho-mqtt OK
```
```

Après correction, le simulateur a affiché :

```
```text
Connexion MQTT réussie.
Message publié.
```
```

---

#### ## 14. Livrables de la tâche 4

Les livrables produits pour cette tâche sont :

- `flow\_node\_red\_ai\_alert.json`
- `README\_integration\_ia.md`
- `capture\_flow\_node\_red\_ai\_alert.png`
- `capture\_prediction\_normal.png`
- `capture\_prediction\_suspect.png`
- `capture\_alert\_danger.png`
- `capture\_mqtt\_publish\_simulator.png`

---

#### ## 15. Conclusion

La tâche 4 permet de connecter Node-RED au modèle IA Python afin de classifier le risque d'incendie en temps réel.

Le flow final reçoit les données MQTT, appelle `predict.py`, enrichit le message avec la prédiction IA et déclenche automatiquement l'envoi d'un message MQTT.

Cette tâche constitue la liaison entre la partie IA et la partie cloud/dashboard de la tâche 5.

...

## logs/sample\_output.txt

Type: text | Source: /mnt/e/projects/iot-fire-detection-ai/logs/sample\_output.txt

FILE: logs/sample\_output.txt

```
```text
Démarrage du simulateur IoT - Membre 1
Broker MQTT : localhost:1883
Topic MQTT  : iot/fire/sensor/data
Intervalle  : 3 secondes
```
```

MQTT inaccessible ([Errno 101] Network is unreachable) : passage en mode hors ligne.

Message généré (mode hors ligne) :

```
{ "device_id": "fire_sensor_01", "timestamp": "2026-05-26T20:10:00", "temperature": 27.4, "smoke": 120, "gas": 180, "humidity": 45 }
```

Message généré (mode hors ligne) :

```
{ "device_id": "fire_sensor_01", "timestamp": "2026-05-26T20:10:03", "temperature": 72.6, "smoke": 840, "gas": 690, "humidity": 65 }
```

...

## simulator/config.json

Type: text | Source: /mnt/e/projects/iot-fire-detection-ai/simulator/config.json

```
FILE: simulator/config.json
```json
{
  "broker_host": "localhost",
  "broker_port": 1883,
  "mqtt_topic": "iot/fire/sensor/data",
  "device_id": "fire_sensor_01",
  "location": "Salle_1",
  "publish_interval_seconds": 3,
  "offline_if_mqtt_unreachable": true,
  "scenario_weights": {
    "normal": 0.65,
    "suspect": 0.25,
    "danger": 0.1
  }
}
```

simulator/simulate_fire_sensors.py

Type: text | Source: /mnt/e/projects/iot-fire-detection-ai/simulator/simulate_fire_sensors.py

```
FILE: simulator/simulate_fire_sensors.py
```python
"""
Projet IoT & IA - Détection intelligente d'incendie
Membre 1 : Simulation des capteurs et publication MQTT

Version corrigée :
- Si le broker MQTT public est inaccessible, le script ne s'arrête plus.
- Il passe automatiquement en mode hors ligne et continue à générer les messages JSON.
- Si paho-mqtt n'est pas installé, le mode hors ligne fonctionne quand même.

Cette partie représente la couche objets / edge du projet.
Elle fournit au Membre 2 un topic MQTT et un format JSON stable pour Node-RED.
"""

import argparse
import json
import random
import socket
import time
from datetime import datetime
from pathlib import Path
from typing import Any, Dict, Optional

try:
 import paho.mqtt.client as mqtt
except ImportError:
 mqtt = None

CONFIG_PATH = Path(__file__).with_name("config.json")

DEFAULT_CONFIG: Dict[str, Any] = {
 "broker_host": "localhost",
 "broker_port": 1883,
 "mqtt_topic": "iot/fire/sensor/data",
 "device_id": "fire_sensor_01",
 "location": "Salle_1",
 "publish_interval_seconds": 3,
 "offline_if_mqtt_unreachable": True,
 "scenario_weights": {
 "normal": 0.65,
 "suspect": 0.25,
 "danger": 0.10
 }
}

def load_config() -> Dict[str, Any]:
 """Load simulator configuration from config.json and merge it with defaults."""
 if not CONFIG_PATH.exists():
 return DEFAULT_CONFIG.copy()

 with CONFIG_PATH.open("r", encoding="utf-8") as file:
```

```

 user_config = json.load(file)

 config = DEFAULT_CONFIG.copy()
 config.update(user_config)

 if "scenario_weights" in user_config:
 merged_weights = DEFAULT_CONFIG["scenario_weights"].copy()
 merged_weights.update(user_config["scenario_weights"])
 config["scenario_weights"] = merged_weights

 return config

def generate_normal_data() -> Dict[str, Any]:
 return {
 "temperature": round(random.uniform(20, 35), 2),
 "smoke": random.randint(20, 200),
 "gas": random.randint(50, 250),
 "humidity": random.randint(45, 65)
 }

def generate_suspect_data() -> Dict[str, Any]:
 return {
 "temperature": round(random.uniform(40, 60), 2),
 "smoke": random.randint(250, 600),
 "gas": random.randint(300, 550),
 "humidity": random.randint(30, 45)
 }

def generate_danger_data() -> Dict[str, Any]:
 return {
 "temperature": round(random.uniform(65, 90), 2),
 "smoke": random.randint(650, 1000),
 "gas": random.randint(600, 900),
 "humidity": random.randint(15, 30)
 }

def choose_scenario(config: Dict[str, Any]) -> str:
 weights = config.get("scenario_weights", DEFAULT_CONFIG["scenario_weights"])
 scenarios = ["normal", "suspect", "danger"]
 probabilities = [
 float(weights.get("normal", 0.65)),
 float(weights.get("suspect", 0.25)),
 float(weights.get("danger", 0.10))
]
 return random.choices(scenarios, weights=probabilities, k=1)[0]

def generate_sensor_message(config: Dict[str, Any]) -> Dict[str, Any]:
 scenario = choose_scenario(config)

 if scenario == "normal":
 values = generate_normal_data()
 elif scenario == "suspect":
 values = generate_suspect_data()
 else:
 values = generate_danger_data()

 return {
 "device_id": config["device_id"],
 "timestamp": datetime.now().isoformat(timespec="seconds"),
 "temperature": values["temperature"],
 "smoke": values["smoke"],
 "gas": values["gas"],
 "humidity": values["humidity"],
 "location": config["location"],
 "scenario": scenario
 }

def on_connect(client, userdata, flags, reason_code, properties=None):
 if reason_code == 0:
 print("Connexion MQTT réussie.")
 else:
 print(f"Connexion MQTT échouée. Code : {reason_code}")

def create_mqtt_client():
 if mqtt is None:
 raise RuntimeError("Le package paho-mqtt n'est pas installé.")

 # Compatible with paho-mqtt 2.x and safe for current versions.
 try:
 client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)

```

```

except Exception:
 client = mqtt.Client()

client.on_connect = on_connect
return client

def connect_mqtt(config: Dict[str, Any]) -> Optional[Any]:
 """Try to connect to MQTT. Return client if OK, otherwise return None."""
 if mqtt is None:
 print("paho-mqtt n'est pas installé : passage en mode hors ligne.")
 return None

 broker_host = config["broker_host"]
 broker_port = int(config["broker_port"])

 try:
 client = create_mqtt_client()
 print(f"Connexion au broker MQTT : {broker_host}:{broker_port}")
 client.connect(broker_host, broker_port)
 client.loop_start()
 return client
 except (OSError, socket.gaierror, TimeoutError, RuntimeError) as exc:
 if config.get("offline_if_mqtt_unreachable", True):
 print(f"MQTT inaccessible ({exc}) : passage en mode hors ligne.")
 return None
 raise

def publish_or_print(client: Optional[Any], topic: str, payload: str) -> None:
 """Publish payload if MQTT is available, otherwise print only."""
 if client is None:
 print("Message généré (mode hors ligne) :")
 print(payload)
 return

 result = client.publish(topic, payload)
 result.wait_for_publish()
 print("Message publié :")
 print(payload)

def parse_args() -> argparse.Namespace:
 parser = argparse.ArgumentParser(description="Simulateur de capteurs incendie - Membre 1")
 parser.add_argument("--offline", action="store_true", help="Forcer le mode hors ligne sans connexion MQTT")
 parser.add_argument("--once", action="store_true", help="Générer un seul message puis arrêter")
 parser.add_argument("--interval", type=int, default=None, help="Intervalle entre deux messages en secondes")
 return parser.parse_args()

def main() -> None:
 args = parse_args()
 config = load_config()

 if args.interval is not None:
 config["publish_interval_seconds"] = args.interval

 broker_host = config["broker_host"]
 broker_port = int(config["broker_port"])
 mqtt_topic = config["mqtt_topic"]
 publish_interval = int(config["publish_interval_seconds"])

 print("Démarrage du simulateur IoT - Membre 1")
 print(f"Broker MQTT : {broker_host}:{broker_port}")
 print(f"Topic MQTT : {mqtt_topic}")
 print(f"Intervalle : {publish_interval} secondes")
 print("-" * 70)

 client = None if args.offline else connect_mqtt(config)

 if args.offline:
 print("Mode hors ligne forcé : aucune connexion MQTT ne sera utilisée.")

 try:
 while True:
 message = generate_sensor_message(config)
 payload = json.dumps(message, ensure_ascii=False)
 publish_or_print(client, mqtt_topic, payload)
 print("-" * 70)

 if args.once:
 break

 time.sleep(publish_interval)
 except KeyboardInterrupt:
 print("Arrêt manuel du simulateur.")

```

```

 finally:
 if client is not None:
 client.loop_stop()
 client.disconnect()
 print("Connexion MQTT fermée.")
 else:
 print("Simulateur arrêté en mode hors ligne.")

if __name__ == "__main__":
 main()
...

```

## tests/mqtt\_subscriber\_test.py

Type: text | Source: /mnt/e/projects/iot-fire-detection-ai/tests/mqtt\_subscriber\_test.py

```

FILE: tests/mqtt_subscriber_test.py
```python
"""
Test de réception MQTT pour le Membre 1.
Utilisation : python tests/mqtt_subscriber_test.py

Important : ce test nécessite une connexion au broker MQTT configuré.
Si votre environnement n'a pas accès à Internet, utilisez un broker Mosquitto local.
"""

import json
from pathlib import Path

import paho.mqtt.client as mqtt

CONFIG_PATH = Path(__file__).resolve().parents[1] / "simulator" / "config.json"

def load_config():
    with CONFIG_PATH.open("r", encoding="utf-8") as file:
        return json.load(file)

def on_connect(client, userdata, flags, reason_code, properties=None):
    config = userdata
    print(f"Connecté au broker. Code : {reason_code}")
    client.subscribe(config["mqtt_topic"])
    print(f"Abonnement au topic : {config['mqtt_topic']}")

def on_message(client, userdata, message):
    payload = message.payload.decode("utf-8")
    print("Message reçu :")
    print(payload)
    try:
        data = json.loads(payload)
        required_fields = ["device_id", "timestamp", "temperature", "smoke", "gas", "humidity", "location", "scenario"]
        missing = [field for field in required_fields if field not in data]
        if missing:
            print(f"Champs manquants : {missing}")
        else:
            print("Format JSON valide pour le Membre 2.")
    except json.JSONDecodeError:
        print("Payload non JSON.")
    print("-" * 70)

def main():
    config = load_config()
    try:
        client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2, userdata=config)
    except Exception:
        client = mqtt.Client(userdata=config)

    client.on_connect = on_connect
    client.on_message = on_message
    client.connect(config["broker_host"], int(config["broker_port"]), 60)
    client.loop_forever()

if __name__ == "__main__":
    main()
...

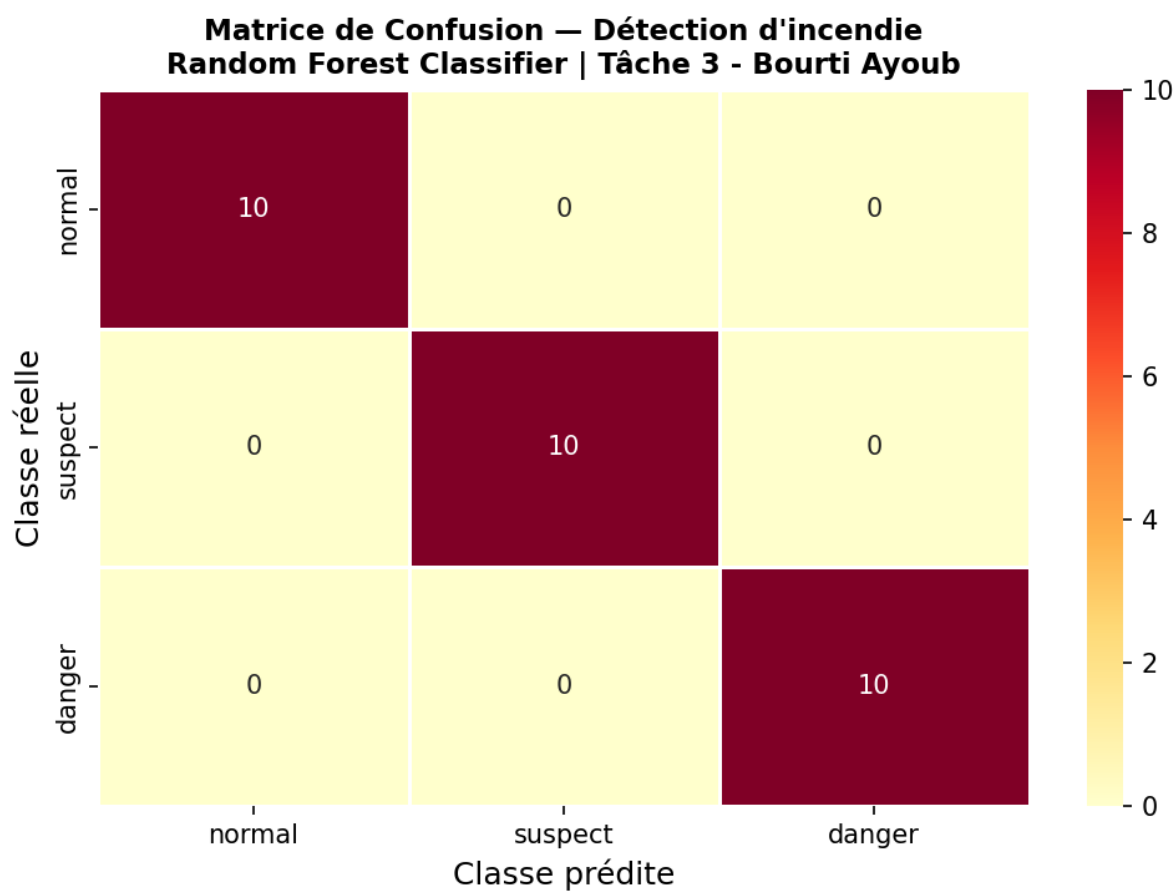
```

3. Images

ai/confusion_matrix.png

Type: image | Source: /mnt/e/projects/iot-fire-detection-ai/ai/confusion_matrix.png

FILE: ai/confusion_matrix.png (embedded preview)



article_screenshots/01_simulator_mqtt_publish.png

Type: image | Source: /mnt/e/projects/iot-fire-detection-ai/article_screenshots/01_simulator_mqtt_publish.png

FILE: article_screenshots/01_simulator_mqtt_publish.png (embedded preview)

```
(youssef) youssef@youssef:/mnt/e/projects/iot-fire-detection-ai$ sudo service mosquitto start
[sudo: authenticate] Password:
(youssef) youssef@youssef:/mnt/e/projects/iot-fire-detection-ai$ python simulator/simulate_fire_sensors.py
Démarrage du simulateur IoT - Membre 1
Broker MQTT : localhost:1883
Topic MQTT : iot/fire/sensor/data
Intervalle : 3 secondes

-----
Connexion au broker MQTT : localhost:1883
Message publié :
{"device_id": "fire_sensor_01", "timestamp": "2026-05-31T16:03:46", "temperature": 27.21, "smoke": 38, "gas": 66, "humidity": 46, "location": "Salle_1", "scenario": "normal"}
-----
Connexion MQTT réussie.
Message publié :
{"device_id": "fire_sensor_01", "timestamp": "2026-05-31T16:03:49", "temperature": 20.9, "smoke": 171, "gas": 205, "humidity": 62, "location": "Salle_1", "scenario": "normal"}
-----
Message publié :
{"device_id": "fire_sensor_01", "timestamp": "2026-05-31T16:03:52", "temperature": 23.09, "smoke": 70, "gas": 106, "humidity": 62, "location": "Salle_1", "scenario": "normal"}
-----
Message publié :
```

article_screenshots/02_mqtt_subscriber_received_json.png

Type: image | Source: /mnt/e/projects/iot-fire-detection-ai/article_screenshots/02_mqtt_subscriber_received_json.png

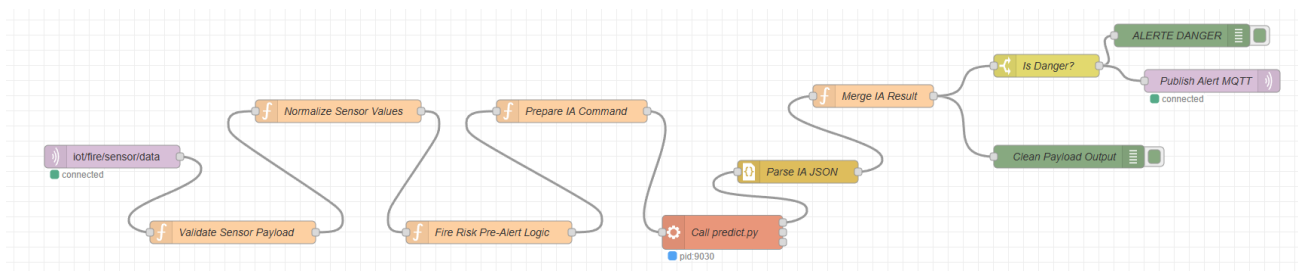
FILE: article_screenshots/02_mqtt_subscriber_received_json.png (embedded preview)

```
(youssef) youssef@youssef:/mnt/e/projects/iot-fire-detection-ai$ mosquitto_sub -h localhost -t iot/fire/sensor/data
{"device_id": "fire_sensor_01", "timestamp": "2026-05-31T16:04:53", "temperature": 31.07, "smoke": 77, "gas": 71, "humidity": 62, "location": "Salle_1", "scenario": "normal"}
{"device_id": "fire_sensor_01", "timestamp": "2026-05-31T16:04:56", "temperature": 25.47, "smoke": 138, "gas": 154, "humidity": 54, "location": "Salle_1", "scenario": "normal"}
{"device_id": "fire_sensor_01", "timestamp": "2026-05-31T16:04:59", "temperature": 84.95, "smoke": 906, "gas": 673, "humidity": 29, "location": "Salle_1", "scenario": "danger"}
{"device_id": "fire_sensor_01", "timestamp": "2026-05-31T16:05:02", "temperature": 54.93, "smoke": 282, "gas": 357, "humidity": 38, "location": "Salle_1", "scenario": "suspect"}
{"device_id": "fire_sensor_01", "timestamp": "2026-05-31T16:05:05", "temperature": 27.62, "smoke": 72, "gas": 89, "humidity": 56, "location": "Salle_1", "scenario": "normal"}
{"device_id": "fire_sensor_01", "timestamp": "2026-05-31T16:05:08", "temperature": 30.55, "smoke": 179, "gas": 84, "humidity": 56, "location": "Salle_1", "scenario": "normal"}
```

article_screenshots/03_node_red_final_flow.png

Type: image | Source: /mnt/e/projects/iot-fire-detection-ai/article_screenshots/03_node_red_final_flow.png

FILE: article_screenshots/03_node_red_final_flow.png (embedded preview)



article_screenshots/04_node_red_debug_normal.png

Type: image | Source: /mnt/e/projects/iot-fire-detection-ai/article_screenshots/04_node_red_debug_normal.png

FILE: article_screenshots/04_node_red_debug_normal.png (embedded preview)

5/31/2026, 4:14:40 PM node: Clean Payload Output

iot/fire/sensor/data : msg.payload : Object

▼ object

device_id: "fire_sensor_01"

timestamp: "2026-05-31T16:14:19"

temperature: 21.61

smoke: 114

gas: 185

humidity: 45

location: "Salle_1"

scenario: "normal"

pre_alert: "normal"

ai_status: "normal"

risk_score: 0.99

alert: false

article_screenshots/05_node_red_debug_suspect.png

Type: image | Source: /mnt/e/projects/iot-fire-detection-ai/article_screenshots/05_node_red_debug_suspect.png

FILE: article_screenshots/05_node_red_debug_suspect.png (embedded preview)

5/31/2026, 4:14:44 PM node: Clean Payload Output

iot/fire/sensor/data : msg.payload : Object

▼ *object*

device_id: "fire_sensor_01"

timestamp: "2026-05-31T16:14:22"

temperature: 41.48

smoke: 511

gas: 474

humidity: 45

location: "Salle_1"

scenario: "suspect"

pre_alert: "suspect"

ai_status: "suspect"

risk_score: 1

alert: false

article_screenshots/06_node_red_debug_danger_alert.png

Type: image | Source: /mnt/e/projects/iot-fire-detection-ai/article_screenshots/06_node_red_debug_danger_alert.png

FILE: article_screenshots/06_node_red_debug_danger_alert.png (embedded preview)

```
5/31/2026, 4:14:37 PM node: ALERTE DANGER
iot/fire/sensor/data : msg.payload : Object
▼ object
  device_id: "fire_sensor_01"
  timestamp: "2026-05-31T16:14:16"
  temperature: 79.35
  smoke: 851
  gas: 836
  humidity: 23
  location: "Salle_1"
  scenario: "danger"
  pre_alert: "danger"
  ai_status: "danger"
  risk_score: 1
  alert: true
```

article_screenshots/07_ai_prediction_tests.png

Type: image | Source: /mnt/e/projects/iot-fire-detection-ai/article_screenshots/07_ai_prediction_tests.png

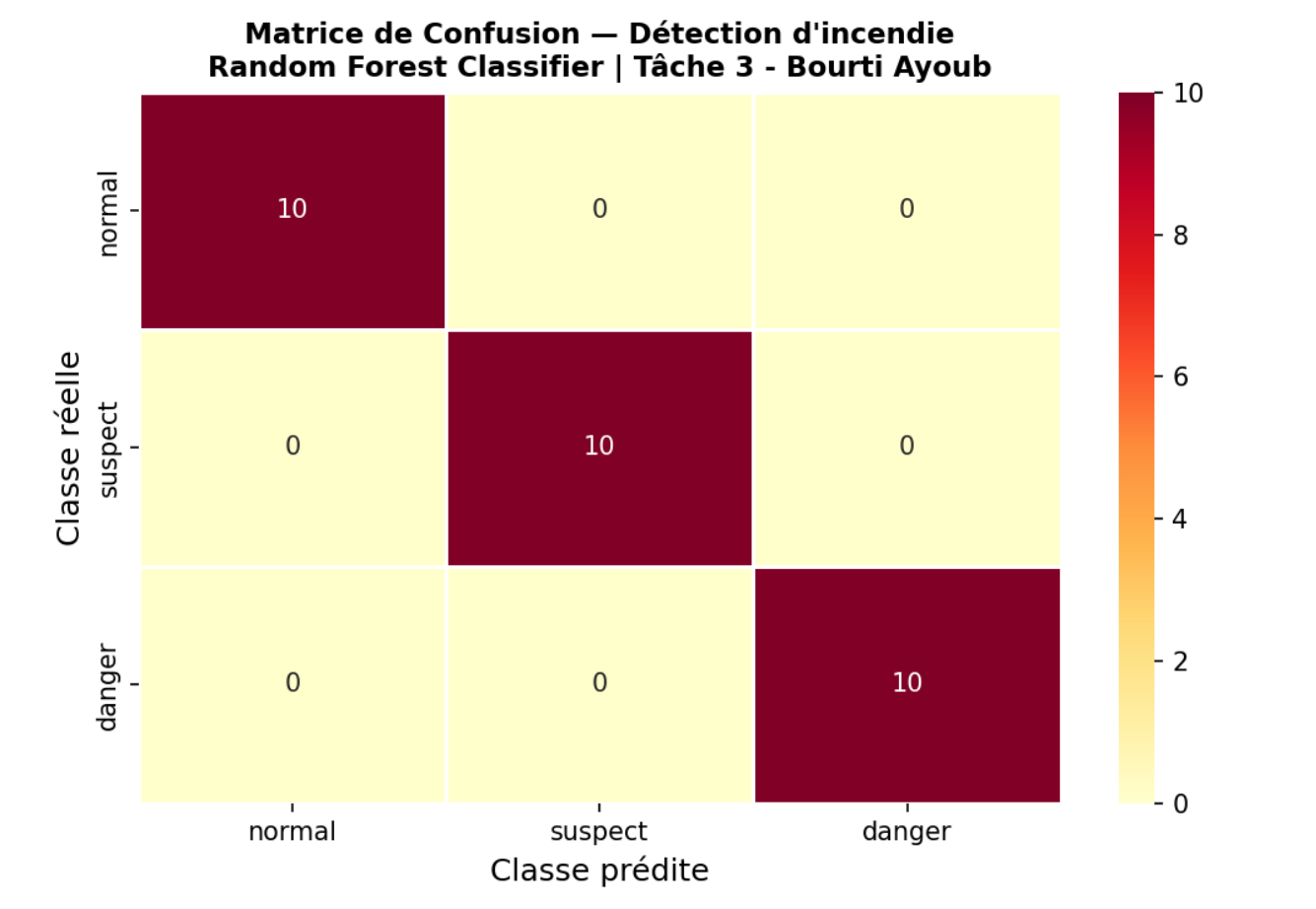
FILE: article_screenshots/07_ai_prediction_tests.png (embedded preview)

```
(youssef) youssef@Youssef:/mnt/e/projects/iot-fire-detection-ai/ai$ cd /mnt/e/projects/iot-fire-detection-ai
python ai/predict.py 24 80 120 55
python ai/predict.py 48 340 300 40
python ai/predict.py 78 850 720 22
{"ai_status": "normal", "risk_score": 1.0, "alert": false}
{"ai_status": "suspect", "risk_score": 1.0, "alert": false}
{"ai_status": "danger", "risk_score": 1.0, "alert": true}
(youssef) youssef@Youssef:/mnt/e/projects/iot-fire-detection-ai$
```

article_screenshots/08_confusion_matrix.png

Type: image | Source: /mnt/e/projects/iot-fire-detection-ai/article_screenshots/08_confusion_matrix.png

FILE: article_screenshots/08_confusion_matrix.png (embedded preview)



article_screenshots/dashboard_danger.png

Type: image | Source: /mnt/e/projects/iot-fire-detection-ai/article_screenshots/dashboard_danger.png

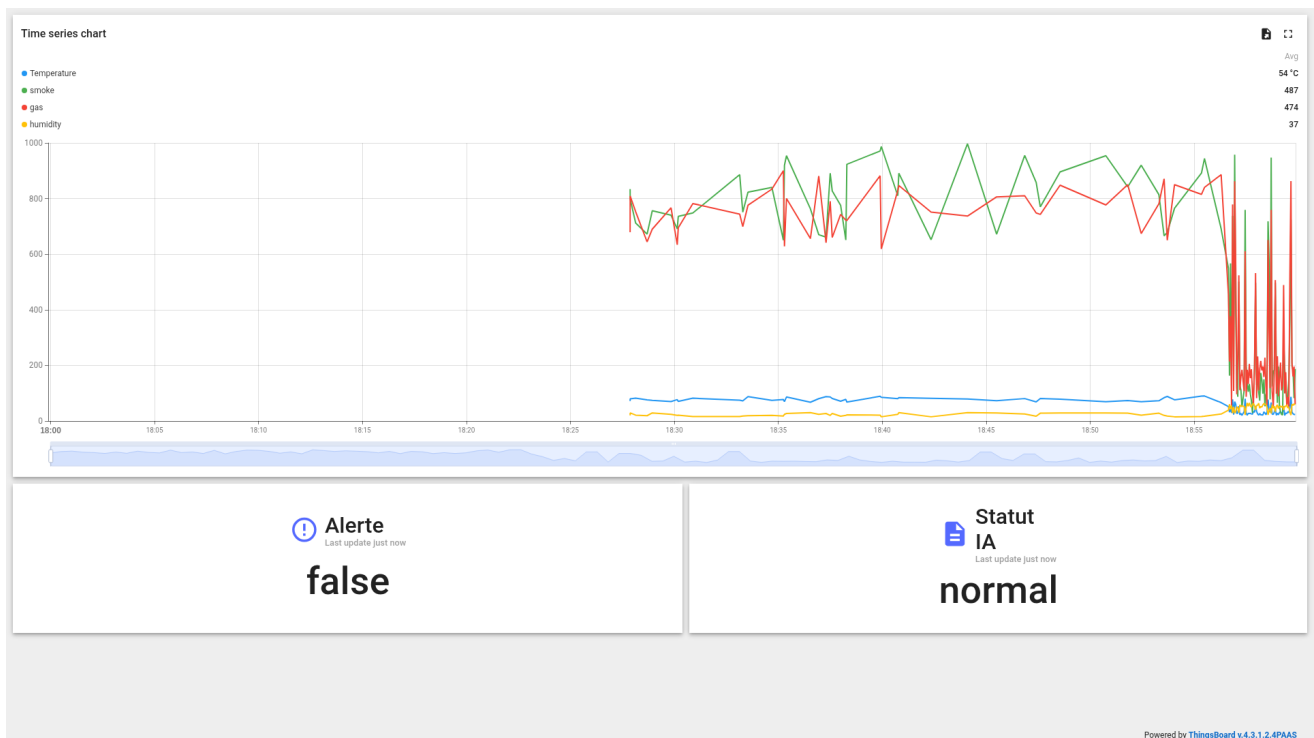
FILE: article_screenshots/dashboard_danger.png (embedded preview)



article_screenshots/dashboard_normal.png

Type: image | Source: /mnt/e/projects/iot-fire-detection-ai/article_screenshots/dashboard_normal.png

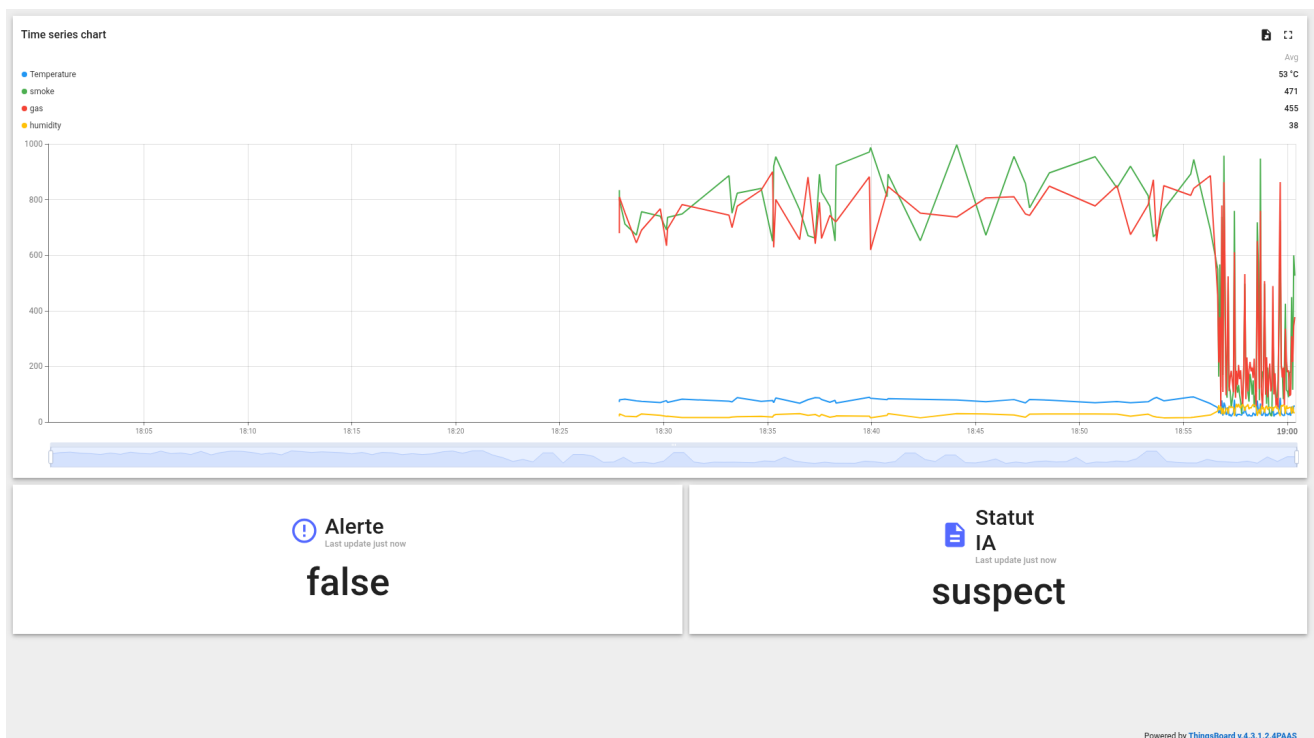
FILE: article_screenshots/dashboard_normal.png (embedded preview)



article_screenshots/dashboard_suspect.png

Type: image | Source: /mnt/e/projects/iot-fire-detection-ai/article_screenshots/dashboard_suspect.png

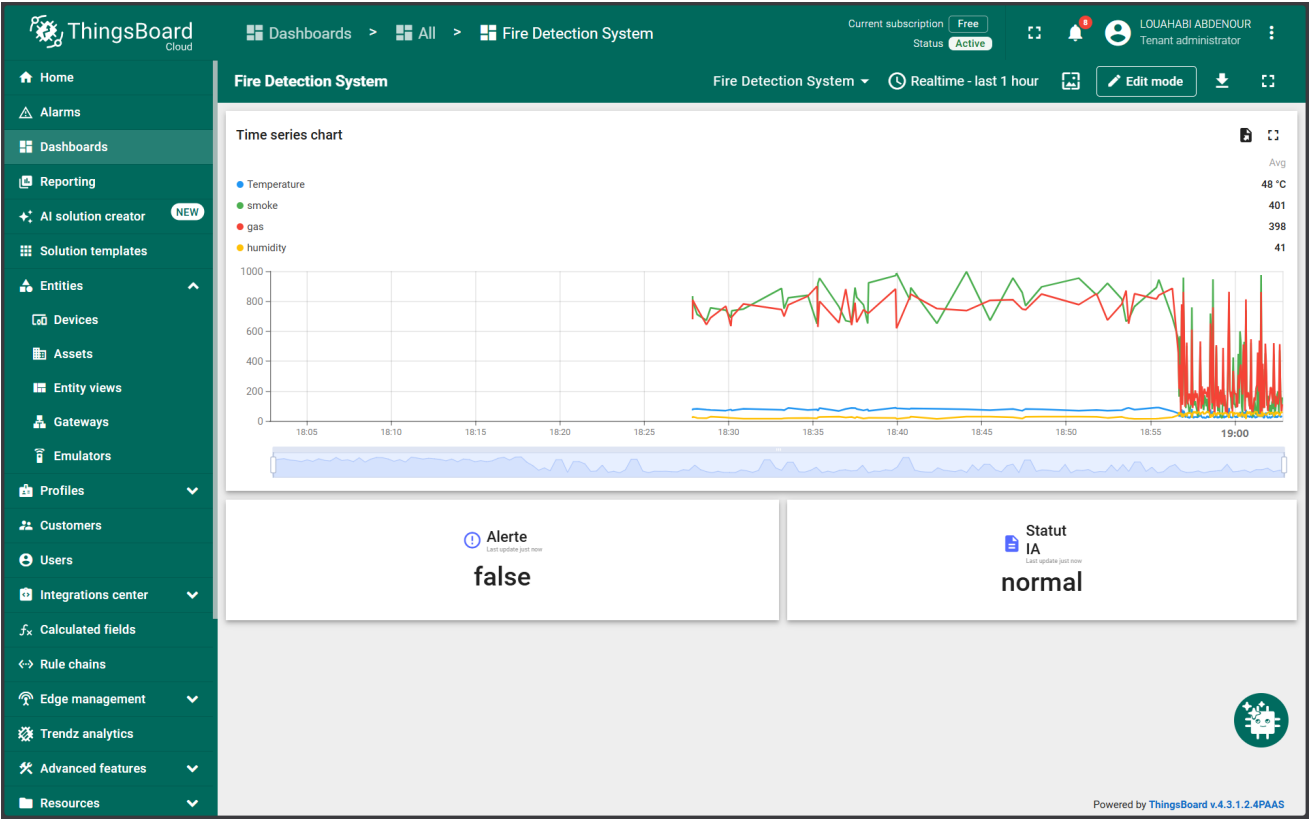
FILE: article_screenshots/dashboard_suspect.png (embedded preview)



article_screenshots/thingsboard_dashboard.png

Type: image | Source: /mnt/e/projects/iot-fire-detection-ai/article_screenshots/thingsboard_dashboard.png

FILE: article_screenshots/thingsboard_dashboard.png (embedded preview)

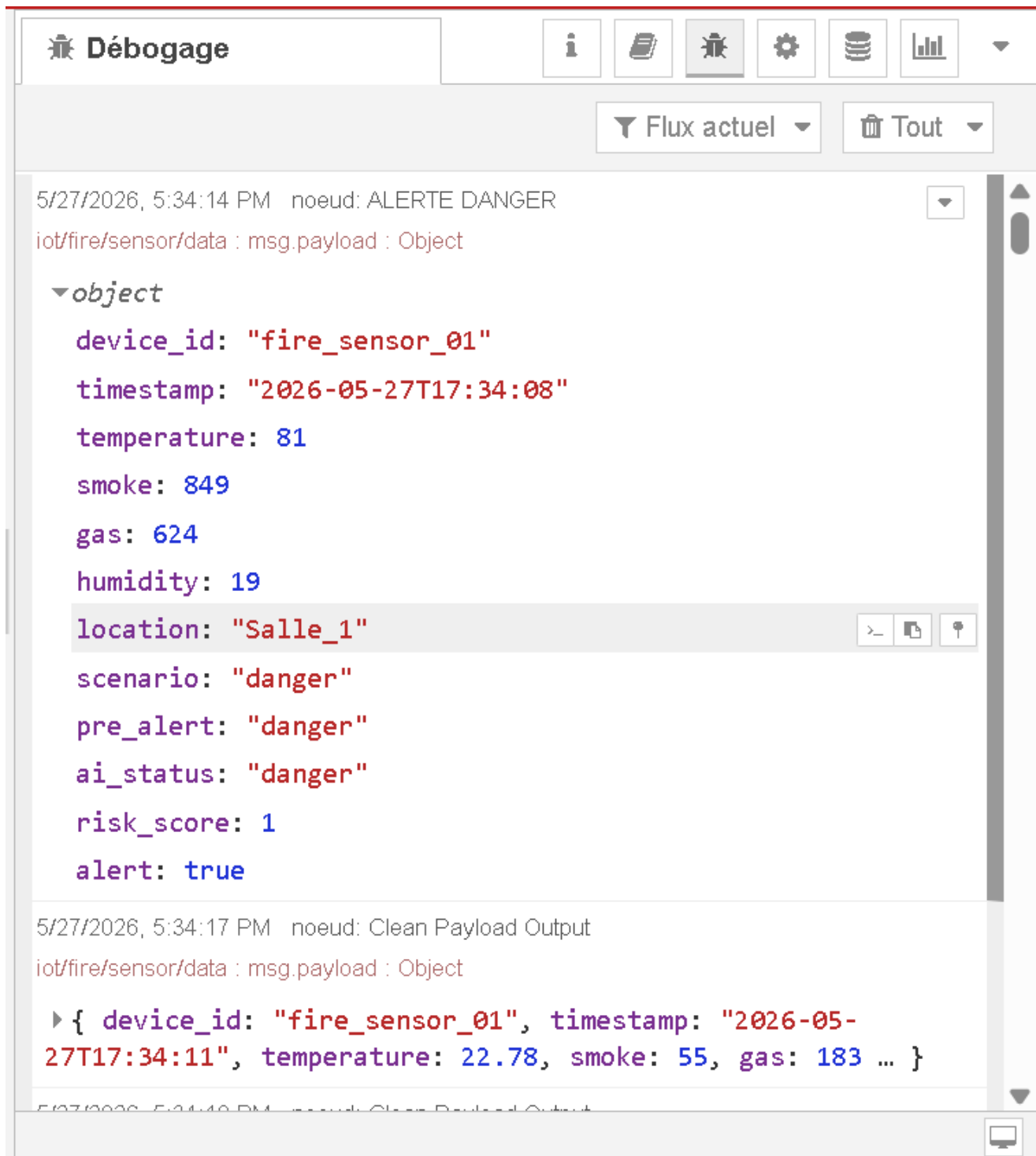


integration_ia_node_red/screenshots_tache4/capture_alert_danger.png

Type: image | Source:

/mnt/e/projects/iot-fire-detection-ai/integration_ia_node_red/screenshots_tache4/capture_alert_danger.png

FILE: integration_ia_node_red/screenshots_tache4/capture_alert_danger.png (embedded preview)

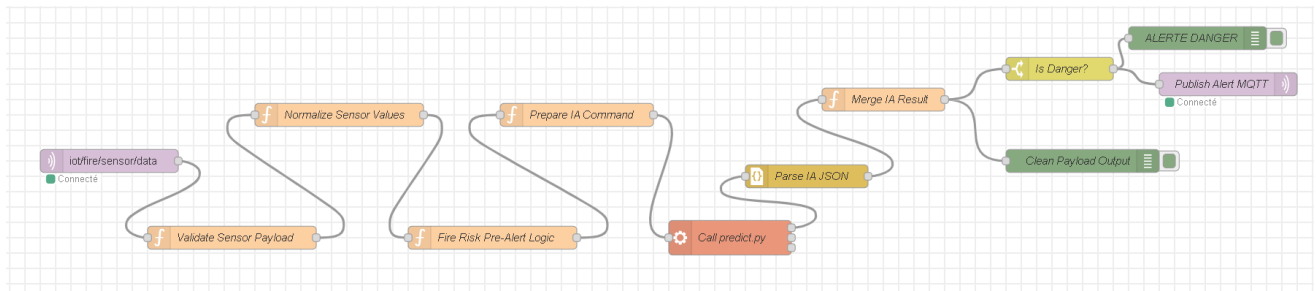


integration_ia_node_red/screenshots_tache4/capture_flow_node_red_ai_alert.png

Type: image | Source:

/mnt/e/projects/iot-fire-detection-ai/integration_ia_node_red/screenshots_tache4/capture_flow_node_red_ai_alert.png

FILE: integration_ia_node_red/screenshots_tache4/capture_flow_node_red_ai_alert.png (embedded preview)



integration_ia_node_red/screenshots_tache4/capture_mqtt_publish_simulator.png

Type: image | Source:

/mnt/e/projects/iot-fire-detection-ai/integration_ia_node_red/screenshots_tache4/capture_mqtt_publish_simulator.png

FILE: integration_ia_node_red/screenshots_tache4/capture_mqtt_publish_simulator.png (embedded preview)

```
C:\Users\USER\Desktop\iot-fire-detection-ai\iot-fire-detection-ai>python simulator\simulate_fire_sensors.py
Démarrage du simulateur IoT - Membre 1
Broker MQTT : localhost:1883
Topic MQTT : iot/fire/sensor/data
Intervalle : 3 secondes

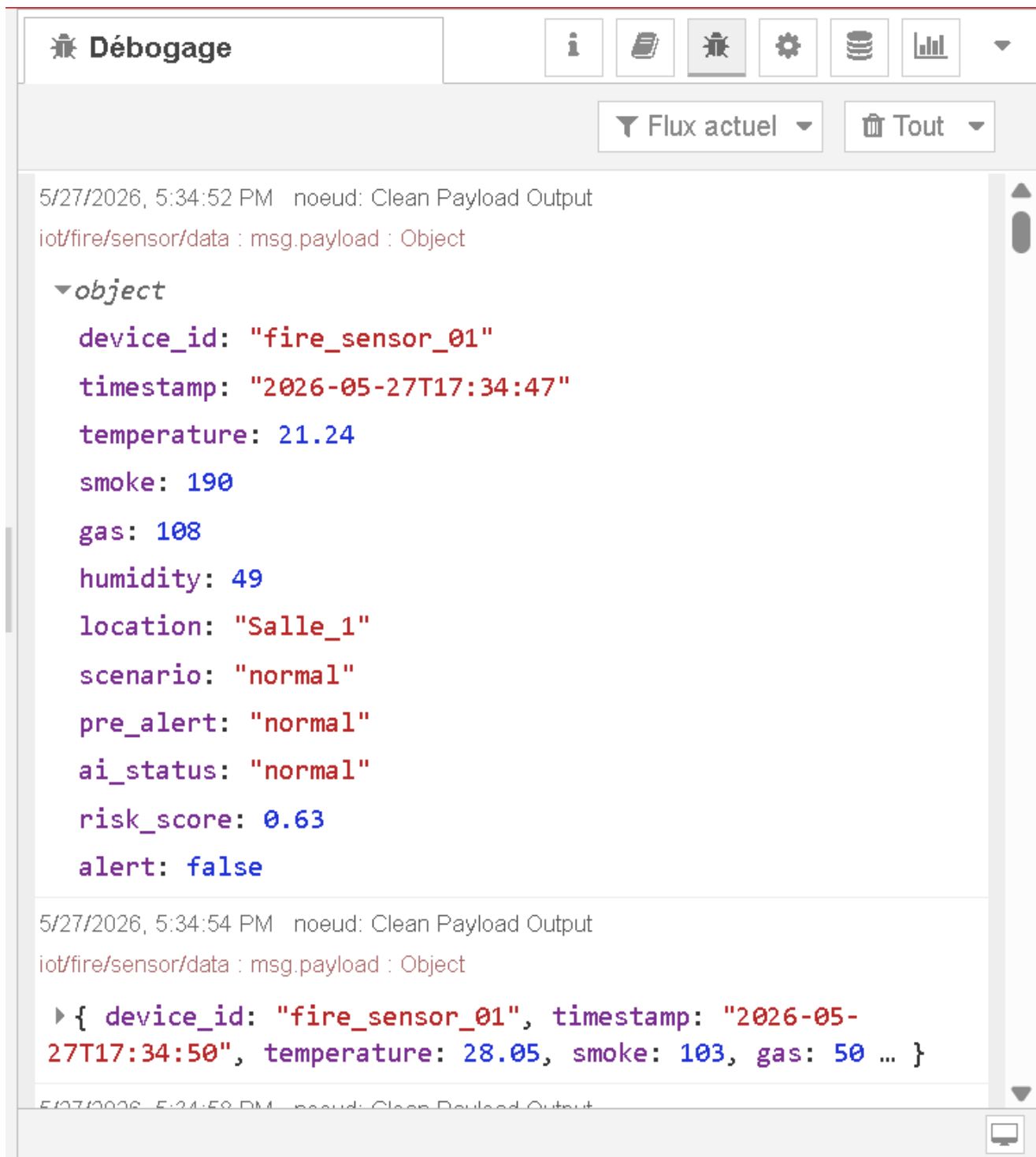
-----
-----
paho-mqtt n'est pas installé : passage en mode hors ligne.
Message généré (mode hors ligne) :
{"device_id": "fire_sensor_01", "timestamp": "2026-05-27T17:20:04", "temperature": 27.97, "smoke": 47, "gas": 148, "humidity": 54, "location": "Salle_1", "scenario": "normal"}
-----
-----
Message généré (mode hors ligne) :
{"device_id": "fire_sensor_01", "timestamp": "2026-05-27T17:20:07", "temperature": 51.36, "smoke": 556, "gas": 392, "humidity": 36, "location": "Salle_1", "scenario": "suspect"}
-----
-----
Message généré (mode hors ligne) :
{"device_id": "fire_sensor_01", "timestamp": "2026-05-27T17:20:10", "temperature": 71.26, "smoke": 712, "gas": 647, "humidity": 15, "location": "Salle_1", "scenario": "danger"}
-----
-----
Message généré (mode hors ligne) :
{"device_id": "fire_sensor_01", "timestamp": "2026-05-27T17:20:13", "temperature": 66.04, "smoke": 988, "gas": 771, "humidity": 21, "location": "Salle_1", "scenario": "danger"}
```

integration_ia_node_red/screenshots_tache4/capture_prediction_normal.png

Type: image | Source:

/mnt/e/projects/iot-fire-detection-ai/integration_ia_node_red/screenshots_tache4/capture_prediction_normal.png

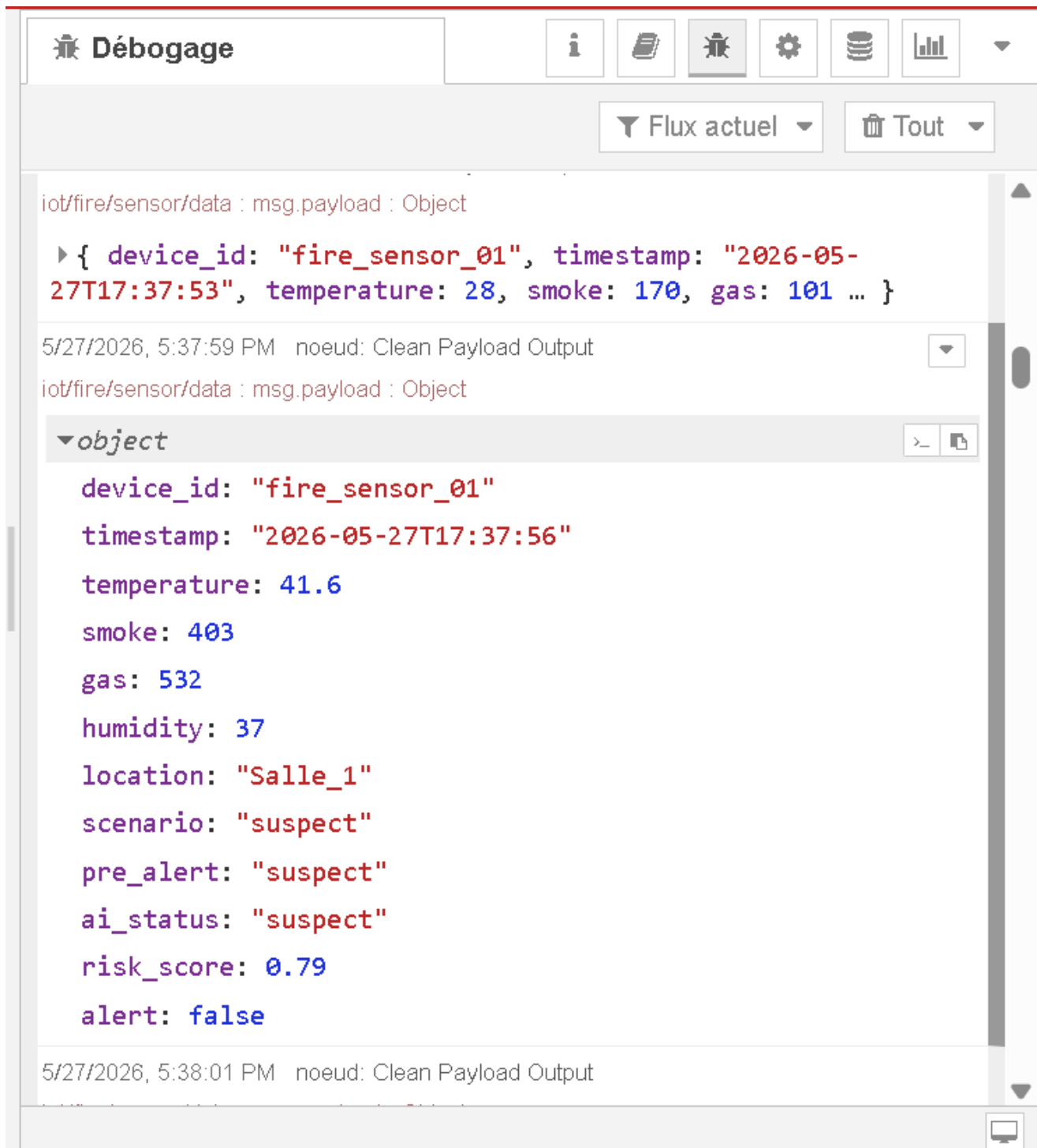
FILE: integration_ia_node_red/screenshots_tache4/capture_prediction_normal.png (embedded preview)



integration_ia_node_red/screenshots_tache4/capture_prediction_suspect.png

Type: image | Source:

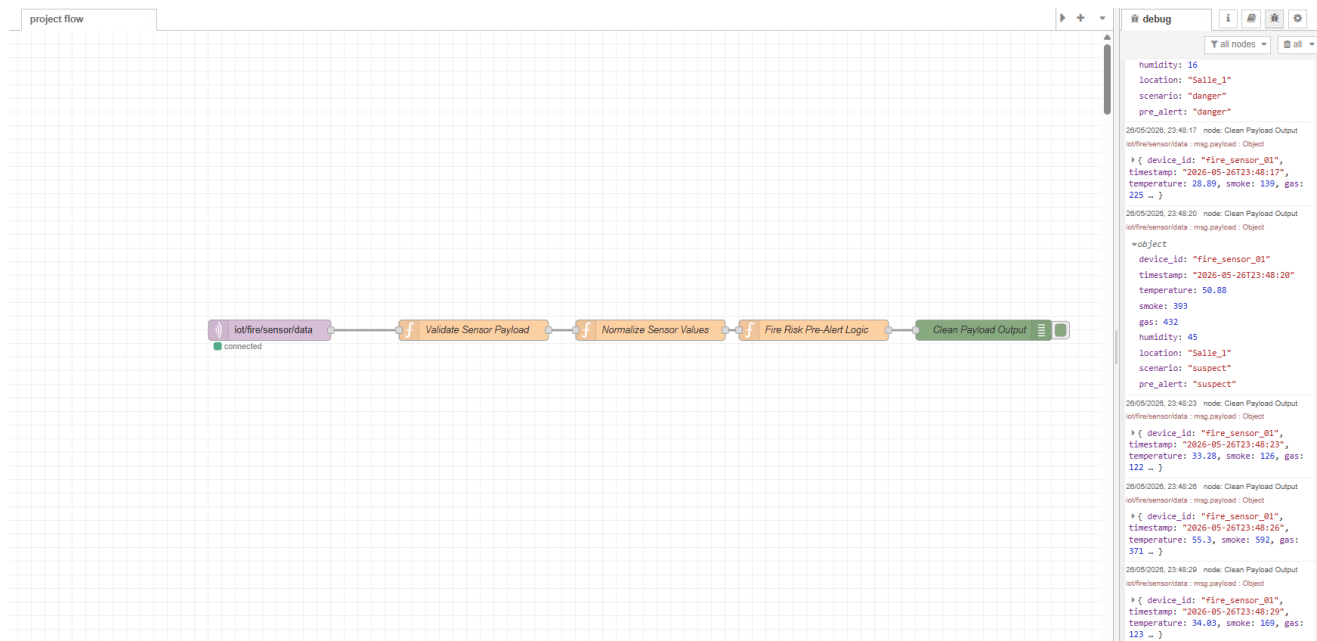
/mnt/e/projects/iot-fire-detection-ai/integration_ia_node_red/screenshots_tache4/capture_prediction_suspect.png



screenshots/capture_node_red_reception.png

Type: image | Source: /mnt/e/projects/iot-fire-detection-ai/screenshots/capture_node_red_reception.png

FILE: screenshots/capture_node_red_reception.png (embedded preview)



screenshots/danger alert.png

Type: image | Source: /mnt/e/projects/iot-fire-detection-ai/screenshots/danger alert.png

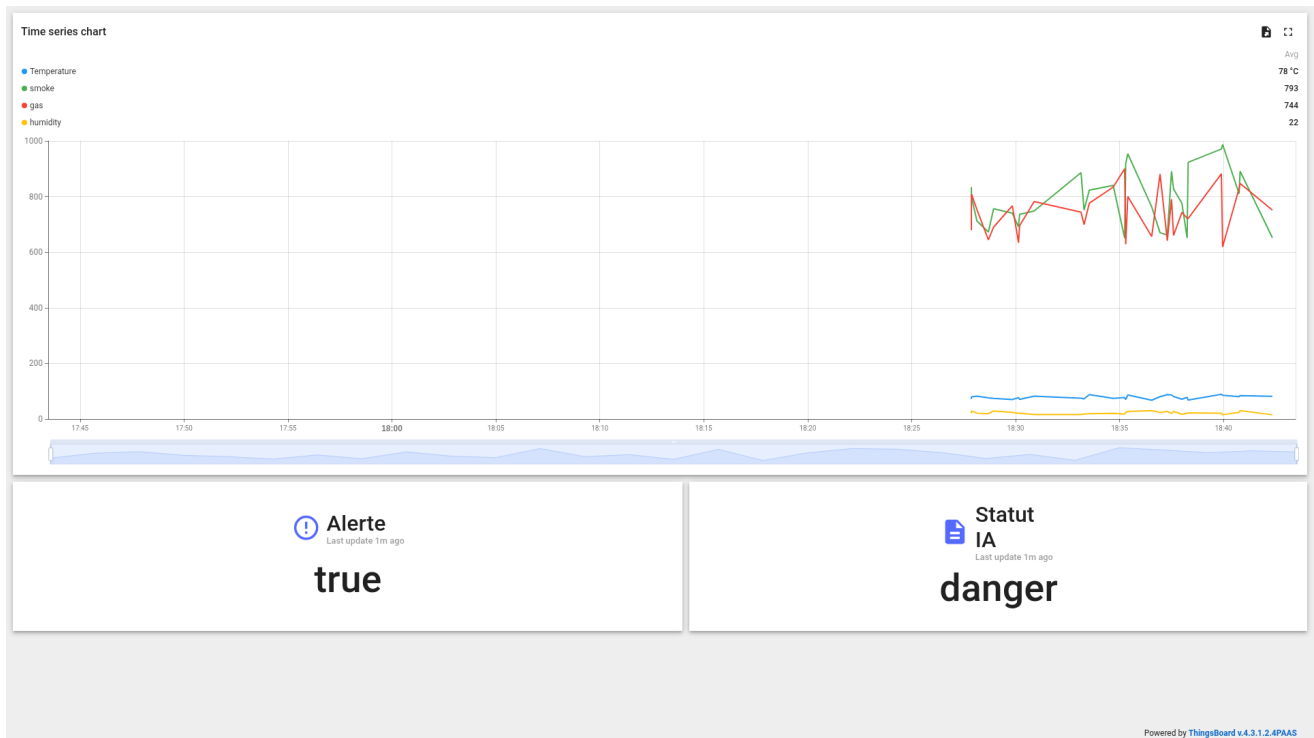
FILE: screenshots/danger alert.png (embedded preview)

```
26/05/2026, 23:47:41 node: Clean Payload Output
iot/fire/sensor/data : msg.payload : Object
{
  device_id: "fire_sensor_01"
  timestamp: "2026-05-26T23:47:41"
  temperature: 69.72
  smoke: 787
  gas: 753
  humidity: 25
  location: "Salle_1"
  scenario: "danger"
  pre_alert: "danger"
}
```

screenshots/dashboard_danger.png

Type: image | Source: /mnt/e/projects/iot-fire-detection-ai/screenshots/dashboard_danger.png

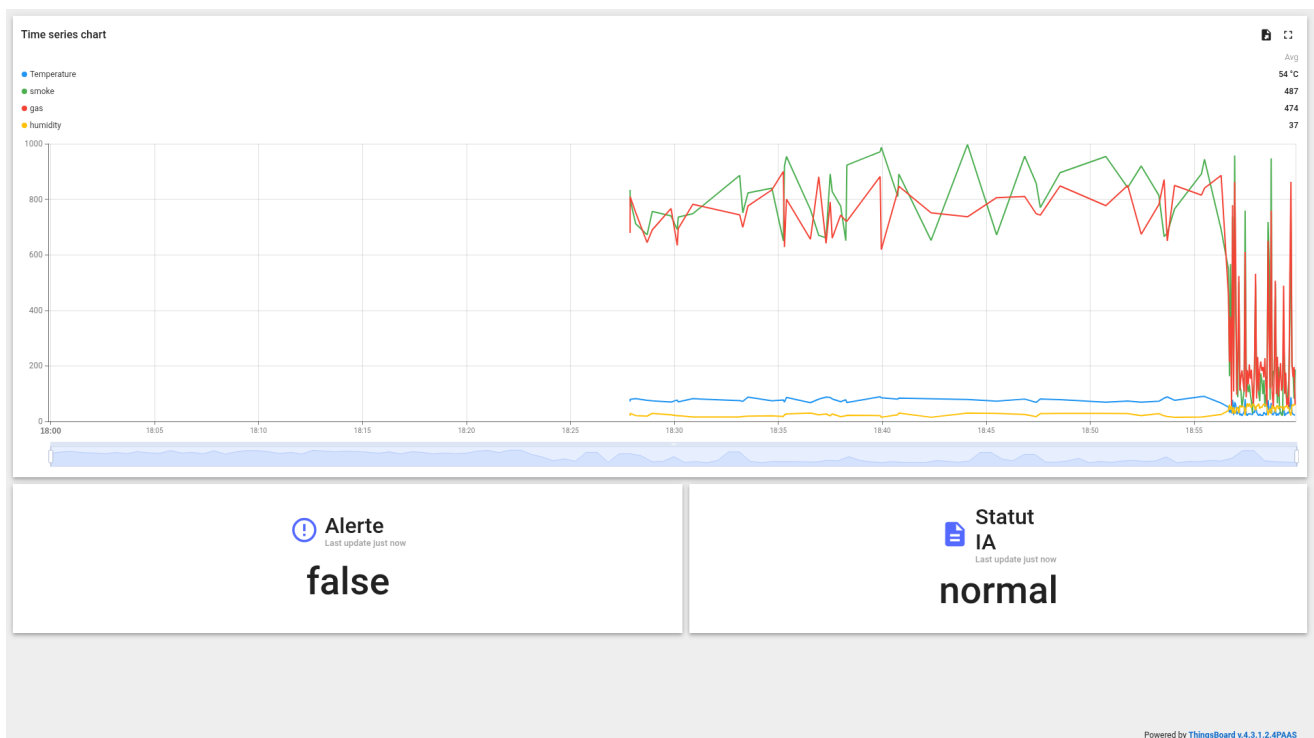
FILE: screenshots/dashboard_danger.png (embedded preview)



screenshots/dashboard_normal.png

Type: image | Source: /mnt/e/projects/iot-fire-detection-ai/screenshots/dashboard_normal.png

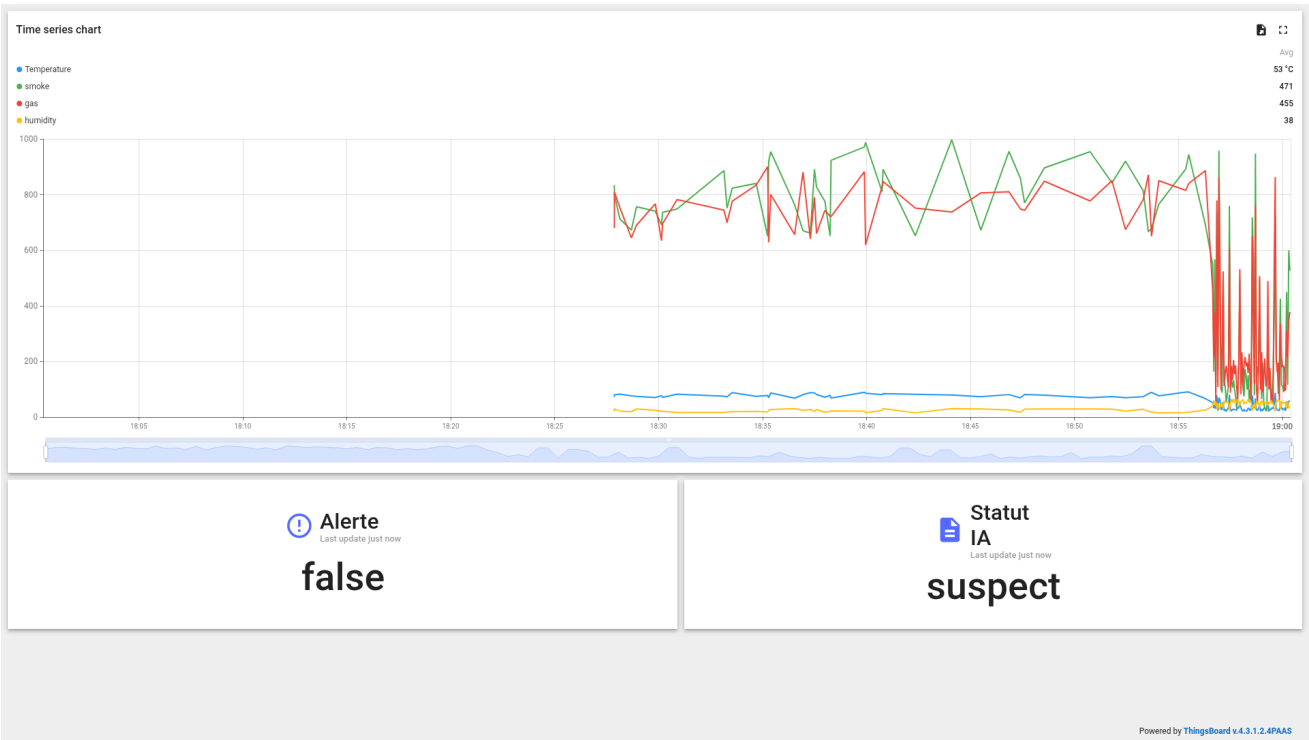
FILE: screenshots/dashboard_normal.png (embedded preview)



screenshots/dashboard_suspect.png

Type: image | Source: /mnt/e/projects/iot-fire-detection-ai/screenshots/dashboard_suspect.png

FILE: screenshots/dashboard_suspect.png (embedded preview)



screenshots/normal alert.png

Type: image | Source: /mnt/e/projects/iot-fire-detection-ai/screenshots/normal alert.png

FILE: screenshots/normal alert.png (embedded preview)

```
26/05/2026, 23:47:29 node: Clean Payload Output
iot/fire/sensor/data : msg.payload : Object
  ▼object
    device_id: "fire_sensor_01"
    timestamp: "2026-05-26T23:47:29"
    temperature: 33.37
    smoke: 133
    gas: 137
    humidity: 47
    location: "Salle_1"
    scenario: "normal"
    pre_alert: "normal"
```

screenshots/suspect alert.png

Type: image | Source: /mnt/e/projects/iot-fire-detection-ai/screenshots/suspect alert.png

FILE: screenshots/suspect alert.png (embedded preview)

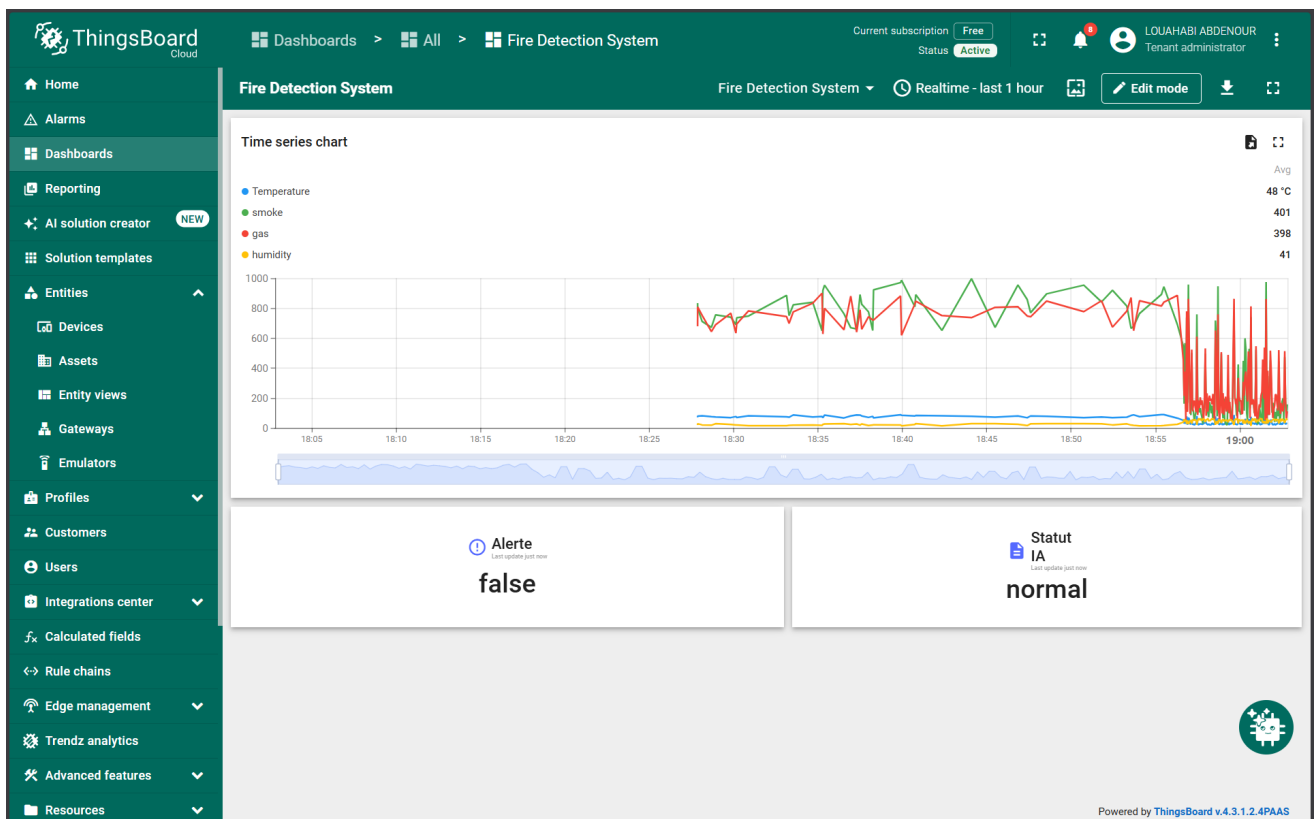
```
26/05/2026, 23:48:20  node: Clean Payload Output
iot/fire/sensor/data : msg.payload : Object

▼object
  device_id: "fire_sensor_01"
  timestamp: "2026-05-26T23:48:20"
  temperature: 50.88
  smoke: 393
  gas: 432
  humidity: 45
  location: "Salle_1"
  scenario: "suspect"
  pre_alert: "suspect"
```

screenshots/thingsboard_dashboard.png

Type: image | Source: /mnt/e/projects/iot-fire-detection-ai/screenshots/thingsboard_dashboard.png

FILE: screenshots/thingsboard_dashboard.png (embedded preview)



4. Skipped Binary Files

- `iot-fire-detection-ai.pdf` [application/pdf]
- `ai/model_fire_detection.joblib` [binary/unknown]

5. Export Summary

- Included files count: 50
- Skipped files count: 2
- Skipped binary files: 2
- Skipped large files: 0
- Warning: no image embed failures